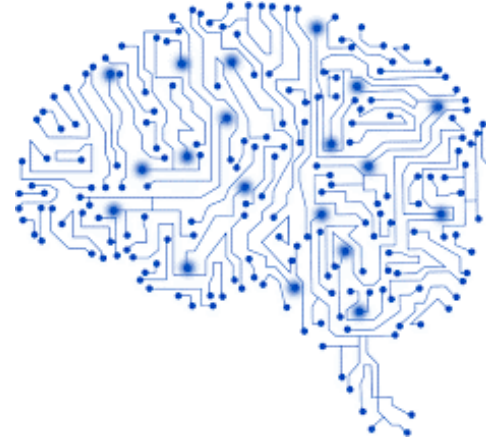




University of Minho
School of Engineering



Dados e Aprendizagem Automática

Unsupervised Learning:
K-means, K-medoids and DBSCAN

DAA 2025/2026

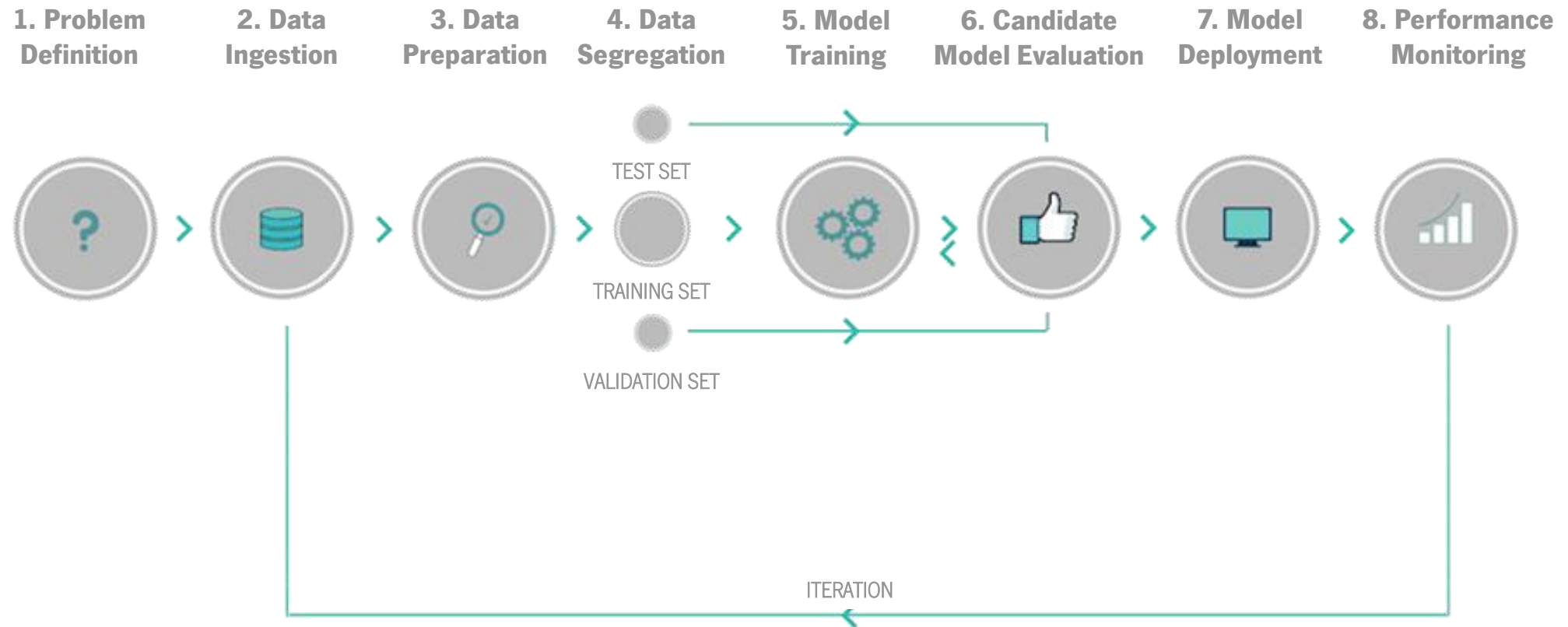
Filipa Ferraz, Victor Alves, Dalila Durães, Francisco Marcondes, Guilherme Barbosa

Part IX

Contents

2

- Unsupervised Learning
 - K-means Clustering
 - K-medoids Clustering
 - DBSCAN Clustering
- Hands On





Unsupervised Learning

Unsupervised Learning

5

In **unsupervised learning**, there is no outcome to be predicted; the algorithm simply tries to identify patterns in the data. **Clustering** involves grouping (clustering) data based on similarity.

Problem: development of a Machine Learning model that can **group** data.

Model: clustering algorithms.

Dataset: isotropic Gaussian blobs generated from `sklearn.datasets`.

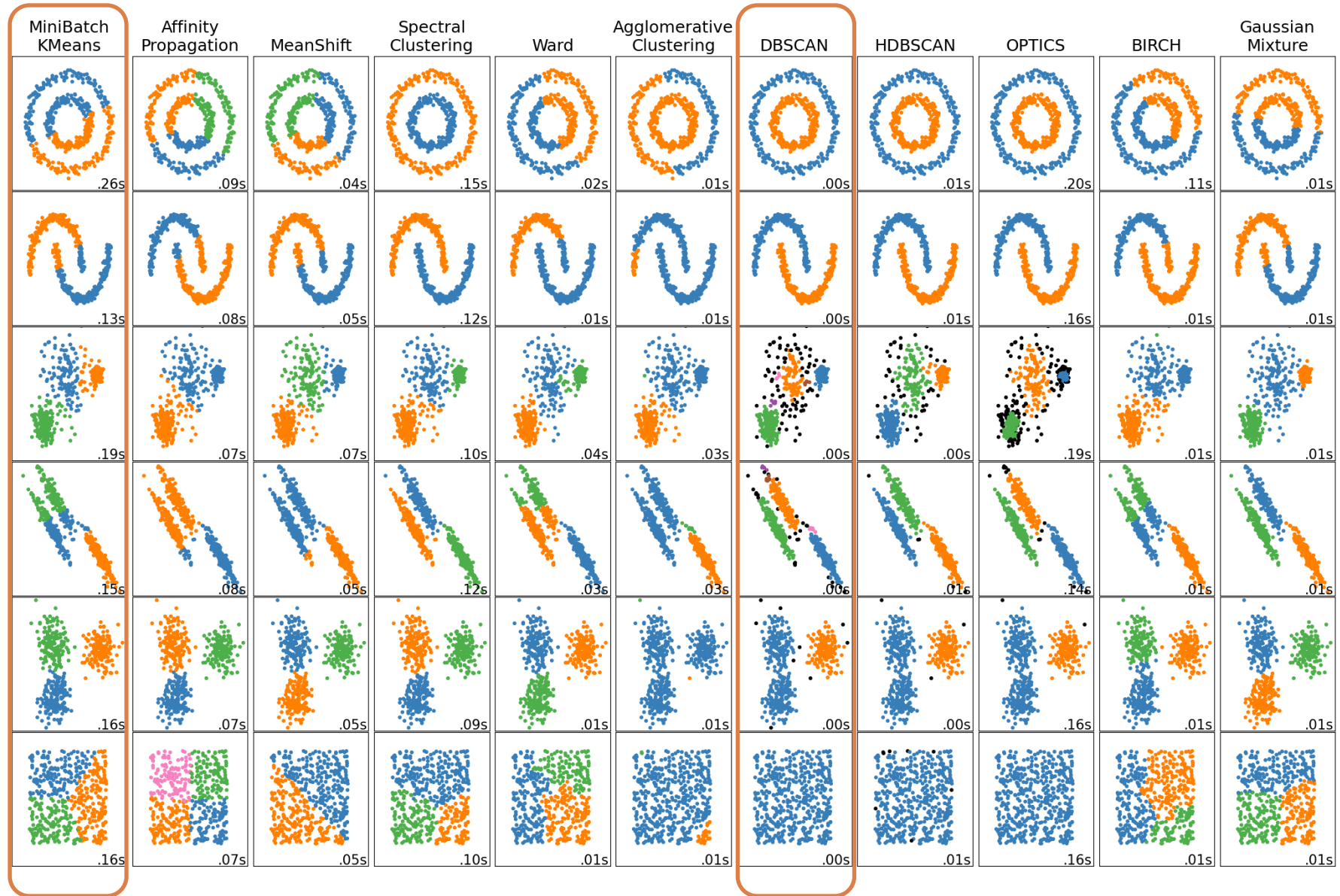
Note: all clustering algorithms require data preprocessing (e.g., dimensionality reduction) and standardisation.

You may need to install [scikit-learn-extra](https://github.com/scikit-learn-contrib/scikit-learn-extra).

```
pip install https://github.com/scikit-learn-contrib/scikit-learn-extra/archive/master.zip
```

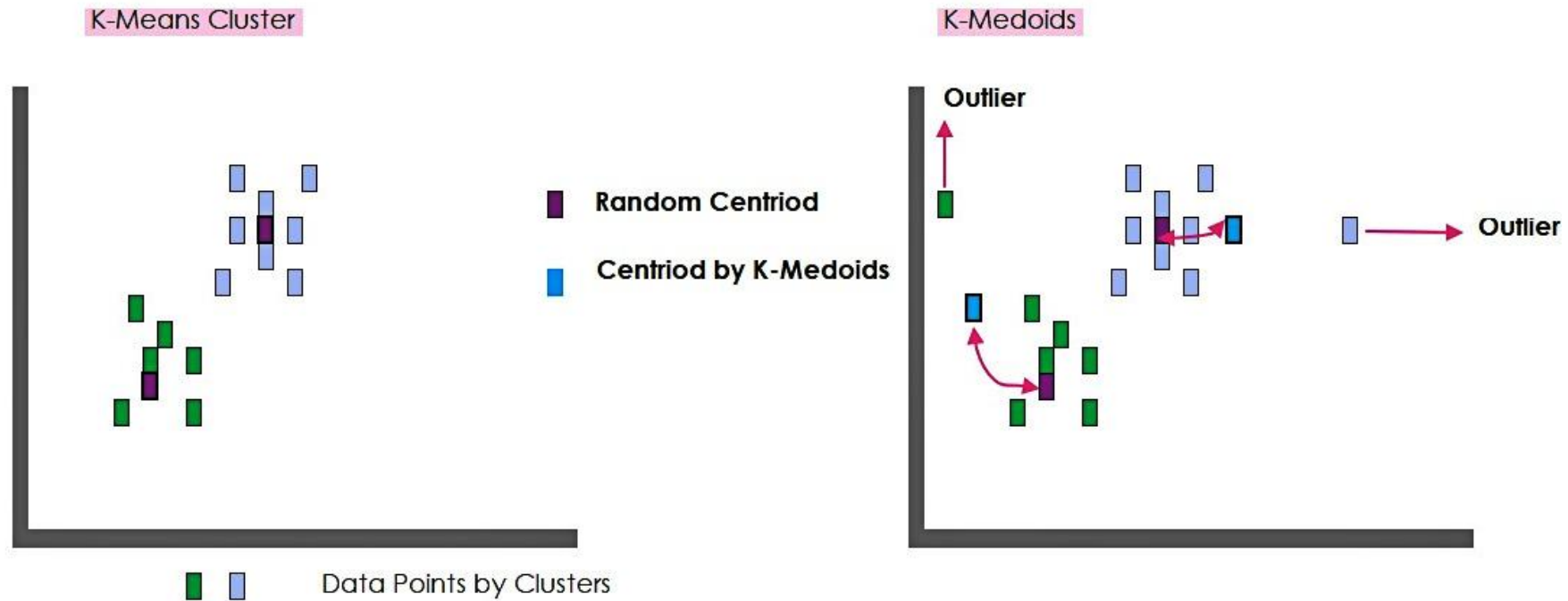
Unsupervised Learning - Clustering

6



K-Means and K-Medoids

7

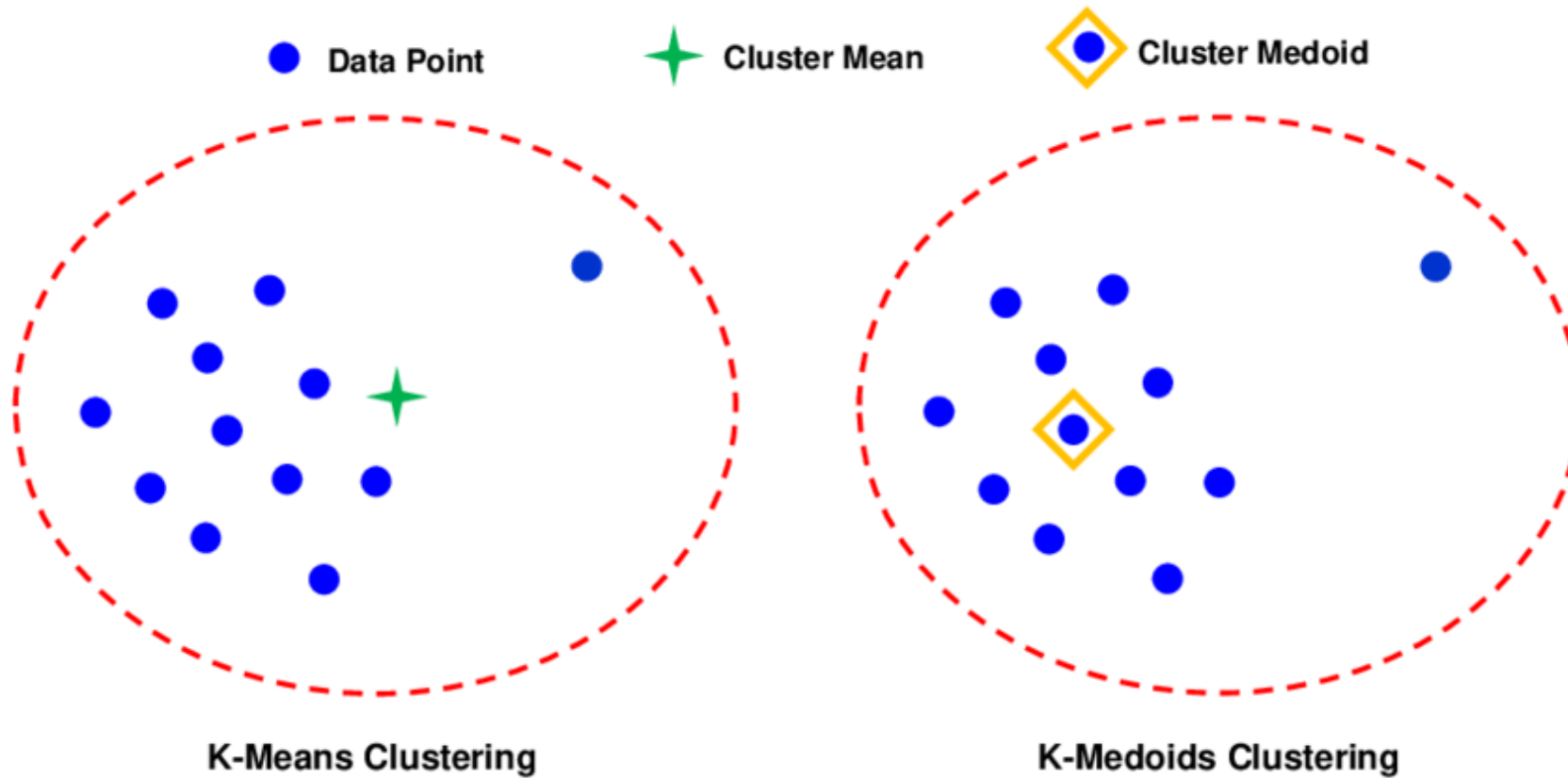


The Choice of Centriod in K-Means is random and what if there is presence of Outlier?

K-Medoids solves the issue as it K-Medoids selected the precise centriod among all data points of the corresponding clusters

K-Means and K-Medoids

8



K-Means and K-Medoids

9

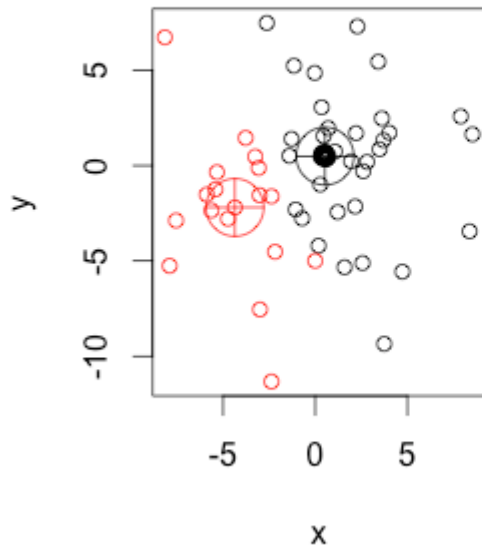
- Both the K-Means and K-Medoids algorithms follow the **partitioning method**. They both break the dataset up into k groups and try to minimise the distance between points in the same cluster and a particular point which is the centre of that cluster.
- The **K-Means** algorithm selects the *centroid*, which is the geometric centre of a cluster, using the mean all the cluster points.
- In contrast, the **K-Medoids** algorithm selects *medoids* - points that act as centres and belong to the dataset - and computes the dissimilarity between data points.
- The most common implementation of the K-Medoids clustering algorithm is the **Partitioning Around Medoids (PAM)** algorithm. However, the PAM algorithm uses a *greedy search*, which may not find the global optimum solution.

K-Means and K-Medoids

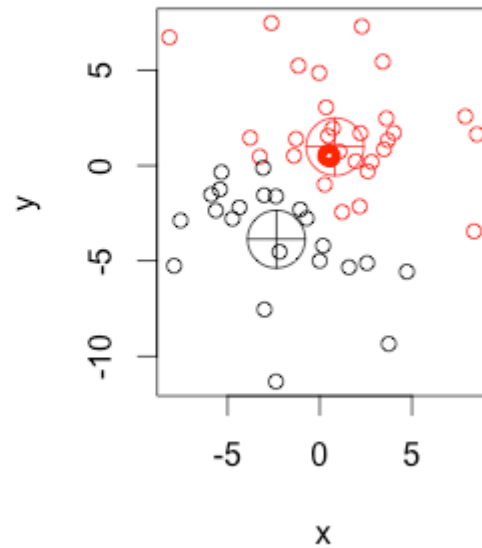
10

1st iteration

Kmedoids Cluster

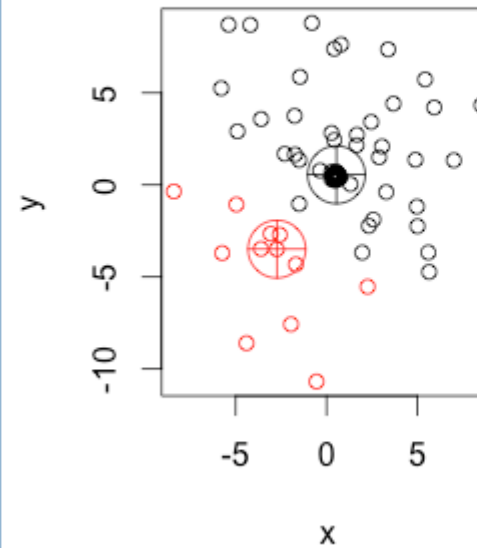


Kmeans Cluster

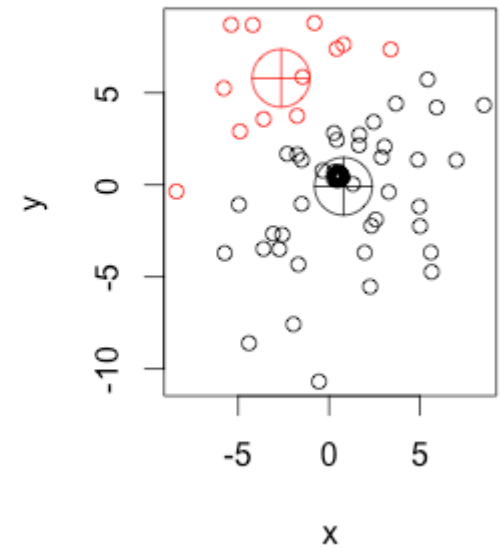


2nd iteration

Kmedoids Cluster

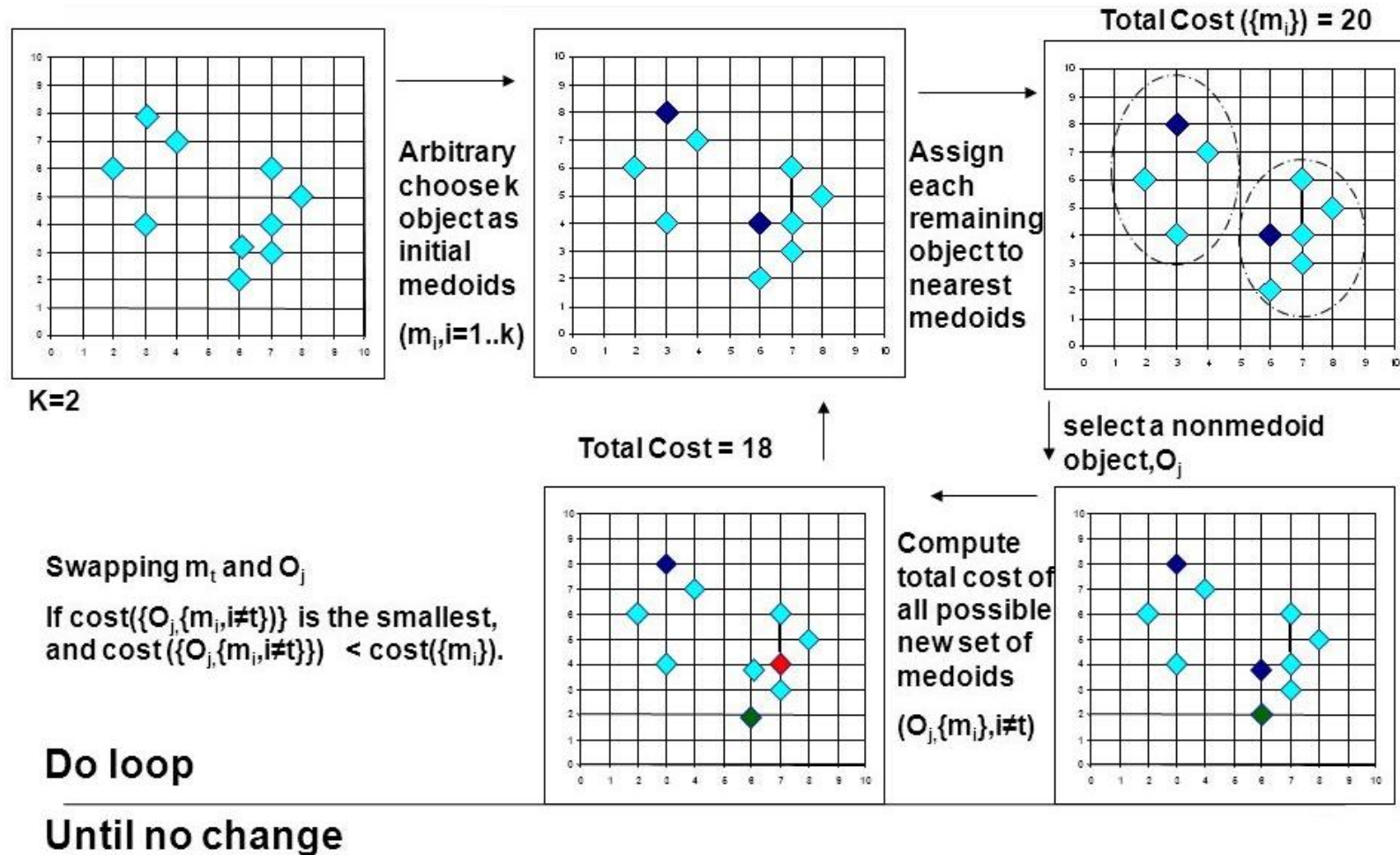


Kmeans Cluster



Partitioning Around Medoids

11



K-Means and K-Medoids

12

Medoids are more robust to outliers than **centroids** but require more computation for high-dimensional data.

	K-Means	K-Medoids
<i>Cluster representation</i>	Uses <u>centroids</u> to represent clusters	Uses <u>medoids</u> to represent clusters
<i>Distance metrics</i>	Only optimises <u>squared Euclidean distance</u>	Can use <u>any distance</u> metric
<i>Computational efficiency</i>	<u>Linear</u> in the size of the dataset if the number of iterations is fixed	The runtime of most algorithms is <u>quadratic</u> in the size of the dataset
<i>Outlier sensitivity</i>	<u>Highly sensitive</u> to noise and outliers	Less sensitive to outliers and noise
<i>Cluster shape</i>	Not suitable for clusters of arbitrary shape	<u>Best suited</u> to <u>clusters</u> that are <u>normally distributed and isotropic</u>

K-Means and K-Medoids

13



Iris Dataset

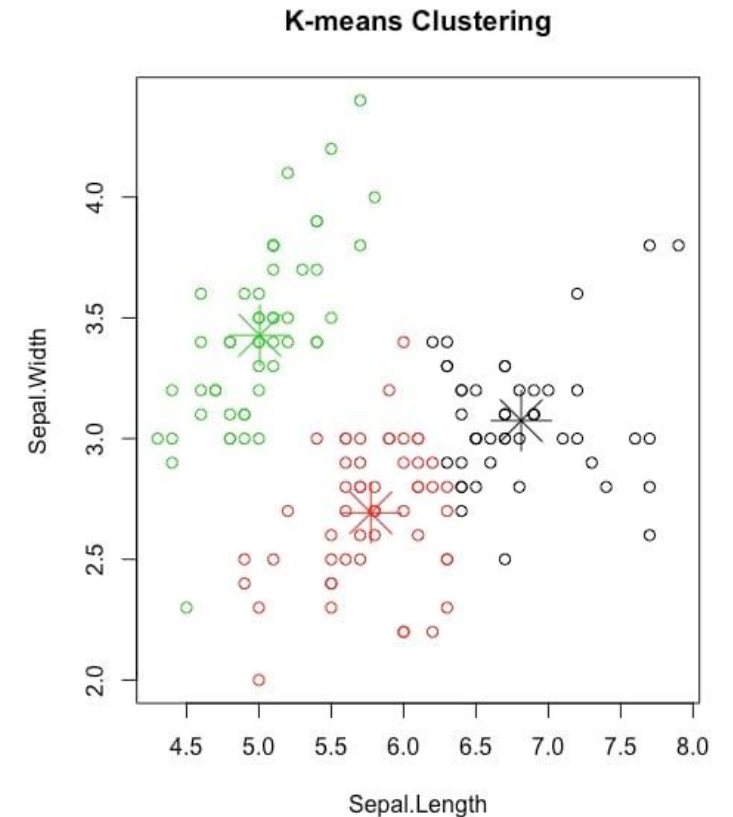
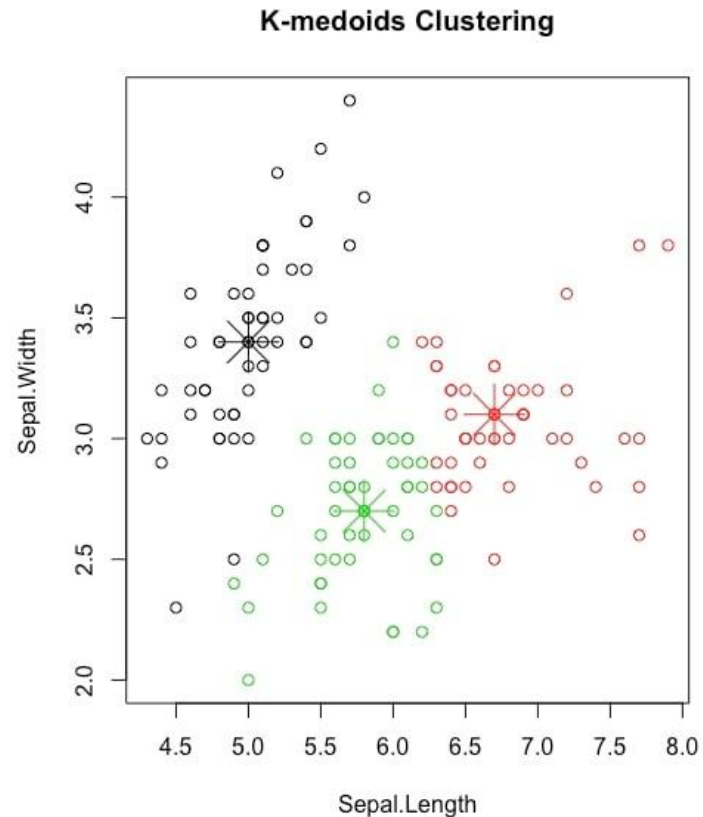
Three species: *Iris setosa*, *Iris versicolour* and *Iris virginica*

150 instances

Four numerical features:

- sepal length (cm) [4.3, 7.9]
- sepal width (cm) [2, 4.4]
- petal length (cm) [1, 6.9]
- petal width (cm) [0.1, 2.5]

```
from sklearn.datasets import load_iris  
  
iris = load_iris()
```



The Data

`sklearn.datasets.make_blobs(n samples=100, n features=2, *, centers=None, cluster std=1.0, center box=(-10.0, 10.0), shuffle=True, random_state=None, return centers=False)`

14

Generate the isotropic Gaussian blobs and create the dataset.

```
from sklearn.datasets import make_blobs
```

```
data = make_blobs(n_samples = 200, n_features = 2, centers = 4, cluster_std = 1.8, random_state = 2022)
```

Define X and y.

```
X = data[0]  
y = data[1]
```

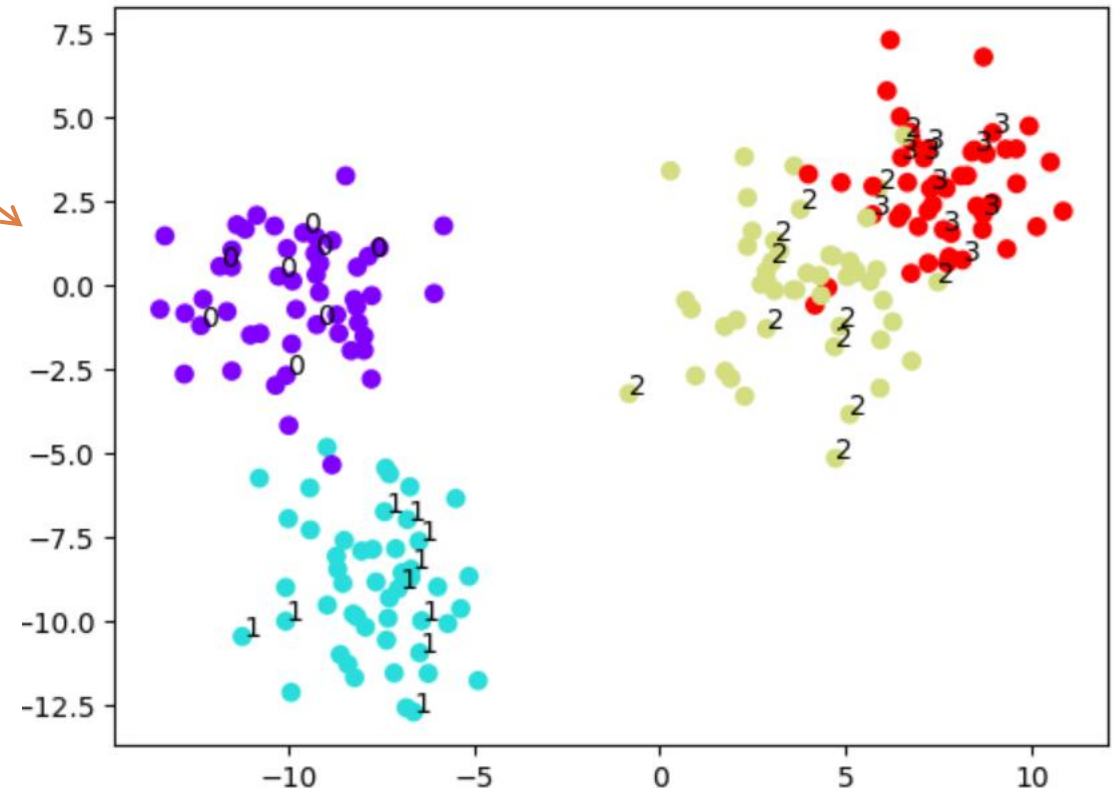
Visualise the data (first 5 instances).

```
print('X:', X[0:5, :])  
print('Y:', y[0:5])
```

```
X: [[ 5.88508997  2.9021639 ]  
 [ -8.20429992 -11.68670283]  
 [  1.9125188  -2.76746603]  
 [ -9.39601207 -7.2830252 ]  
 [  6.1986976   7.32152342]]  
Y: [2 1 2 1 3]
```

```
plt.scatter(X[:, 0], X[:, 1], c = y, cmap = 'rainbow')  
for i, txt in enumerate(y):  
    if i%5 == 0:  
        plt.annotate(txt, (X[i, 0], X[i, 1]))
```

4 groups



K-Means

15

```
from sklearn.cluster import KMeans
```

```
sklearn.cluster.KMeans(n_clusters=8, *, init='k-means++', n_init='auto', max_iter=300,  
tol=0.0001, verbose=0, random_state=None, copy_x=True, algorithm='lloyd')
```

```
kmeans = KMeans(n_clusters = 4, n_init = 10, random_state = 2022)
```

```
kmeans.fit(X)
```

▼ KMeans ⓘ ?

```
KMeans(n_clusters=4, n_init=10, random_state=2022)
```

Obtain the vector at the center of each cluster.

```
kmeans.cluster_centers_
```

```
array([[ -7.68797564, -8.88054369],  
       [  7.70499062,  2.96975295],  
       [-9.78544862, -0.2509739 ],  
       [ 3.60428123, -0.21752545]])
```

```
kmeans.labels_
```

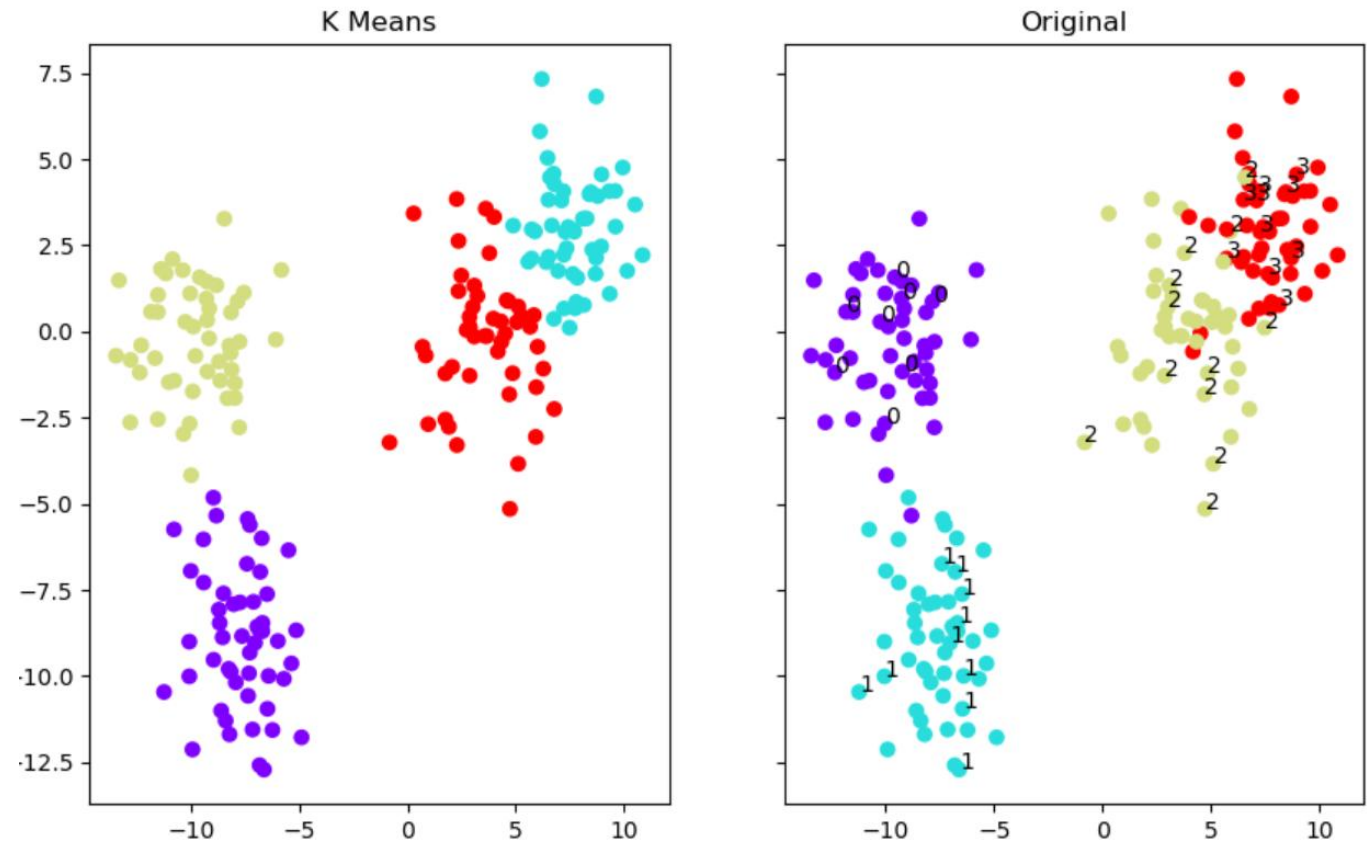
```
array([[1, 0, 3, 0, 1, 2, 0, 2, 3, 3, 1, 1, 3, 1, 2, 1, 1, 1, 1, 2, 0, 3,  
       1, 1, 0, 0, 3, 0, 3, 3, 1, 0, 1, 3, 1, 3, 0, 1, 0, 2, 1, 2, 1, 3,  
       3, 3, 0, 0, 3, 0, 0, 3, 1, 1, 0, 3, 1, 3, 1, 2, 2, 2, 3, 3, 1, 2,  
       0, 2, 2, 2, 3, 2, 3, 0, 0, 2, 2, 0, 0, 2, 1, 1, 2, 3, 0, 0, 1, 3,  
       2, 0, 3, 1, 2, 3, 3, 1, 3, 2, 3, 3, 0, 1, 2, 0, 2, 3, 2, 3, 2, 2,  
       1, 2, 2, 2, 3, 2, 3, 3, 0, 3, 1, 1, 1, 3, 0, 0, 1, 3, 2, 1, 0, 2,  
       2, 0, 0, 0, 2, 2, 1, 0, 0, 0, 1, 3, 0, 1, 0, 2, 3, 2, 3, 3, 1, 1,  
       1, 1, 1, 1, 0, 0, 0, 0, 0, 3, 1, 2, 0, 0, 1, 2, 3, 1, 1, 0, 3, 1,  
       3, 0, 0, 2, 3, 0, 2, 2, 0, 2, 1, 1, 2, 3, 2, 3, 3, 0, 2, 1, 2, 0,  
       2, 2])
```

K-Means

16

Visualise the clusters.

```
f, (ax1, ax2) = plt.subplots(1, 2, sharey = True, figsize = (10, 6))
ax1.set_title('K Means')
ax1.scatter(X[:, 0], X[:, 1], c = kmeans.labels_, cmap = 'rainbow')
ax2.set_title("Original")
ax2.scatter(X[:, 0], X[:, 1], c = y, cmap = 'rainbow')
for i, txt in enumerate(y):
    if i%5 == 0:
        plt.annotate(txt, (X[i, 0], X[i, 1]))
```



Note that the colours are meaningless in relation to the two plots.

K-Means

17

Align the K-Means prediction class with the real values.

```
y_pred = kmeans.predict(X)
```

y_pred

```
array([1, 0, 3, 0, 1, 2, 0, 2, 3, 3, 1, 1, 3, 1, 2, 1, 1, 1, 1, 2, 0, 3,
       1, 1, 0, 0, 3, 0, 3, 3, 1, 0, 1, 3, 1, 3, 0, 1, 0, 2, 1, 2, 1, 3,
       3, 3, 0, 0, 3, 0, 0, 3, 1, 1, 0, 3, 1, 3, 1, 2, 2, 2, 3, 3, 1, 2,
       0, 2, 2, 2, 3, 2, 3, 0, 0, 2, 2, 0, 0, 2, 1, 1, 2, 3, 0, 0, 1, 3,
       2, 0, 3, 1, 2, 3, 3, 1, 3, 2, 3, 3, 0, 1, 2, 0, 2, 3, 2, 3, 2, 2,
       1, 2, 2, 2, 3, 2, 3, 3, 0, 3, 1, 1, 1, 3, 0, 0, 1, 3, 2, 1, 0, 2,
       2, 0, 0, 0, 2, 2, 1, 0, 0, 0, 1, 3, 0, 1, 0, 2, 3, 2, 3, 3, 1, 1,
       1, 1, 1, 1, 0, 0, 0, 0, 0, 3, 1, 2, 0, 0, 1, 2, 3, 1, 1, 0, 3, 1,
       3, 0, 0, 2, 3, 0, 2, 2, 0, 2, 1, 1, 2, 3, 2, 3, 3, 0, 2, 1, 2, 0,
       2, 2])
```

y

```
array([2, 1, 2, 1, 3, 0, 1, 0, 2, 2, 3, 3, 2, 3, 0, 3, 3, 3, 3, 0, 1, 2,
       3, 3, 1, 1, 3, 1, 2, 2, 3, 1, 3, 3, 3, 2, 1, 3, 1, 0, 3, 0, 3, 2,
       2, 2, 1, 1, 2, 1, 1, 2, 3, 3, 1, 2, 3, 2, 3, 0, 0, 0, 2, 2, 3, 0,
       1, 0, 0, 0, 2, 0, 2, 1, 1, 0, 0, 1, 1, 0, 2, 3, 0, 2, 1, 1, 3, 2,
       0, 1, 2, 3, 0, 2, 2, 3, 2, 0, 2, 2, 1, 3, 0, 1, 0, 2, 0, 2, 0, 0,
       3, 0, 0, 0, 2, 0, 2, 2, 1, 2, 3, 3, 3, 2, 1, 1, 3, 2, 0, 3, 1, 0,
       0, 1, 1, 1, 0, 0, 3, 1, 1, 1, 3, 2, 1, 3, 1, 0, 2, 0, 2, 2, 3, 3,
       2, 3, 3, 3, 0, 1, 1, 1, 1, 2, 3, 0, 1, 1, 3, 0, 2, 3, 3, 1, 2, 3,
       2, 1, 1, 0, 2, 1, 0, 0, 1, 0, 3, 3, 0, 2, 0, 3, 2, 1, 0, 2, 0, 1,
       0, 0])
```

```
y_pred = np.where(y_pred==0, 10, y_pred)
y_pred = np.where(y_pred==2, 0, y_pred)
y_pred = np.where(y_pred==3, 2, y_pred)
y_pred = np.where(y_pred==1, 3, y_pred)
y_pred = np.where(y_pred==10, 1, y_pred)
```

y_pred

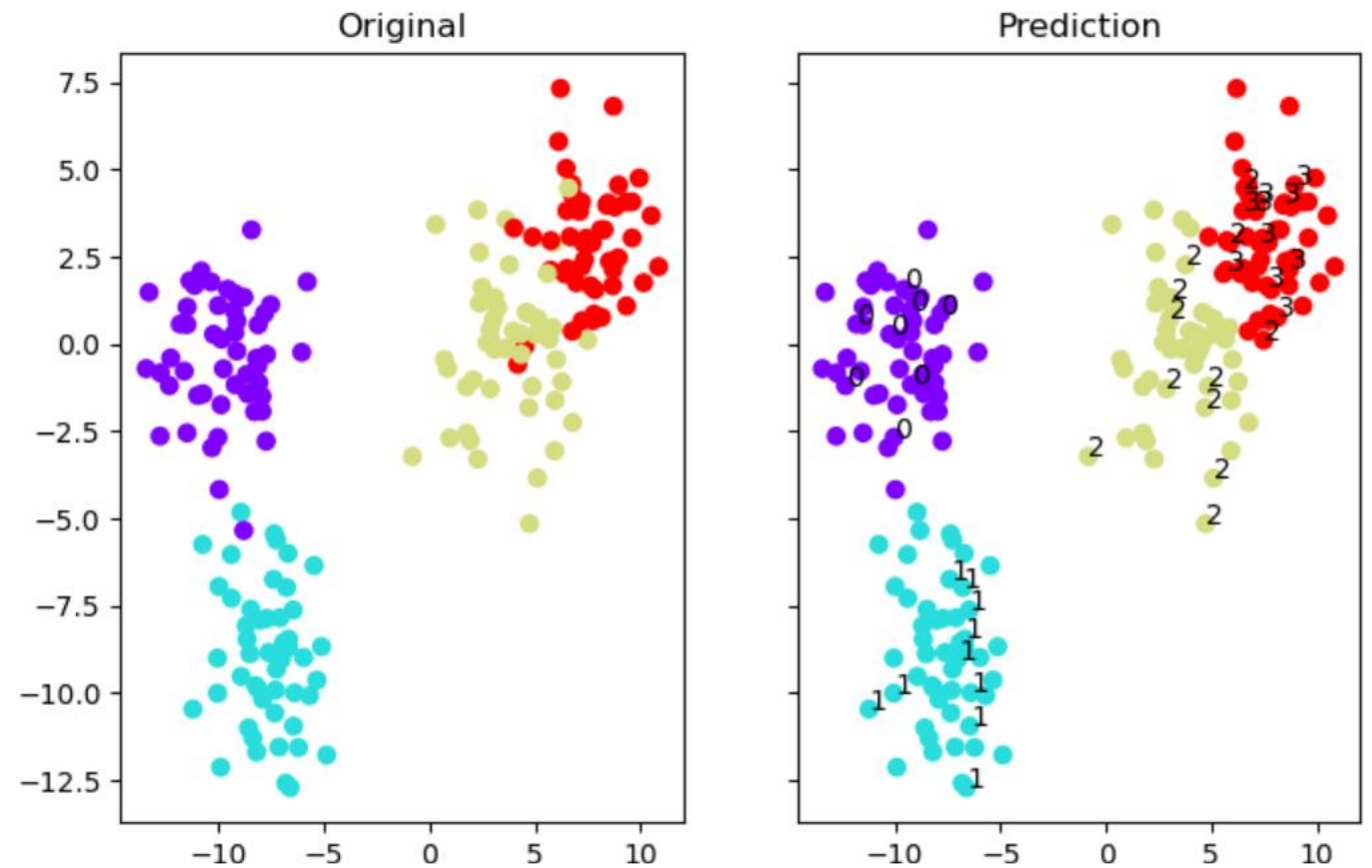
```
array([3, 1, 2, 1, 3, 0, 1, 0, 2, 2, 3, 3, 2, 3, 0, 3, 3, 3, 3, 0, 1, 2,
       3, 3, 1, 1, 2, 1, 2, 2, 3, 1, 3, 2, 3, 2, 1, 3, 1, 0, 3, 0, 3, 2,
       2, 2, 1, 1, 2, 1, 1, 2, 3, 3, 1, 2, 3, 2, 3, 0, 0, 0, 2, 2, 3, 0,
       1, 0, 0, 0, 2, 0, 2, 1, 1, 0, 0, 1, 1, 0, 3, 3, 0, 2, 1, 1, 3, 2,
       0, 1, 2, 3, 0, 2, 2, 3, 2, 0, 2, 2, 1, 3, 0, 1, 0, 2, 0, 2, 0, 0,
       3, 0, 0, 0, 2, 0, 2, 2, 1, 2, 3, 3, 3, 2, 1, 1, 3, 2, 0, 3, 1, 0,
       0, 1, 1, 1, 0, 0, 3, 1, 1, 1, 3, 2, 1, 3, 1, 0, 2, 0, 2, 2, 3, 3,
       3, 3, 3, 3, 1, 1, 1, 1, 1, 2, 3, 0, 1, 1, 3, 0, 2, 3, 3, 1, 2, 3,
       2, 1, 1, 0, 2, 1, 0, 0, 1, 0, 3, 3, 0, 2, 0, 2, 2, 1, 0, 3, 0, 1,
       0, 0])
```

K-Means

18

Redo the visualisations.

```
f, (ax1, ax2) = plt.subplots(1, 2, sharey = True, figsize = (8, 5))
ax1.set_title('Original')
ax1.scatter(X[:,0], X[:,1], c=y, cmap='rainbow')
ax2.set_title("Prediction")
ax2.scatter(X[:,0], X[:,1], c=y_pred, cmap='rainbow')
for i, txt in enumerate(y):
    if i%5 == 0:
        plt.annotate(txt, (X[i,0], X[i,1]))
plt.savefig("KMeans_pred.png")
```



K-Means

19

Evaluate the model.

```
print(confusion_matrix(y, y_pred))
```

```
[[49  1  0  0]
 [ 0 50  0  0]
 [ 0  0 46  4]
 [ 0  0  3 47]]
```

```
print(classification_report(y, y_pred))
```

	precision	recall	f1-score	support
0	1.00	0.98	0.99	50
1	0.98	1.00	0.99	50
2	0.94	0.92	0.93	50
3	0.92	0.94	0.93	50
accuracy			0.96	200
macro avg	0.96	0.96	0.96	200
weighted avg	0.96	0.96	0.96	200

To classify new points, it is best to train a classifier using the clustering result.

More metrics

20

As blobs provides true labels, the following metrics can be used for analysis:

<u>Homogeneity</u>	<code>sklearn.metrics.homogeneity_score(labels_true, labels_pred)</code>
<u>Completeness</u>	<code>sklearn.metrics.completeness_score(labels_true, labels_pred)</code>
<u>V-measure</u>	<code>sklearn.metrics.v_measure_score(labels_true, labels_pred, *, beta=1.0)</code>
<u>Adjusted Rand Index (ARI)</u>	<code>sklearn.metrics.adjusted_rand_score(labels_true, labels_pred)</code>
<u>Adjusted Mutual Information (AMI)</u>	<code>sklearn.metrics.adjusted_mutual_info_score(labels_true, labels_pred, *, average_method='arithmetic')</code>
<u>Silhouette coefficient</u>	<code>sklearn.metrics.silhouette_score(X, labels, *, metric='euclidean', sample_size=None, random_state=None, **kws)</code>

K-Means

21

```
from sklearn import metrics

print(f"Homogeneity: {metrics.homogeneity_score(y, y_pred):.3f}")
print(f"Completeness: {metrics.completeness_score(y, y_pred):.3f}")
print(f"V-measure: {metrics.v_measure_score(y, y_pred):.3f}")
print(f"Adjusted Rand Index: {metrics.adjusted_rand_score(y, y_pred):.3f}")
print(f"Adjusted Mutual Information: {metrics.adjusted_mutual_info_score(y, y_pred):.3f}")
print(f"Silhouette Coefficient: {metrics.silhouette_score(X, y_pred):.3f}")
```

```
Homogeneity: 0.891
Completeness: 0.891
V-measure: 0.891
Adjusted Rand Index: 0.898
Adjusted Mutual Information: 0.889
Silhouette Coefficient: 0.559
```

K-Medoids

22

```
from sklearn_extra.cluster import KMedoids
```

`sklearn_extra.cluster.KMedoids(n_clusters=8, metric='euclidean', method='alternate',
init='heuristic', max_iter=300, random_state=None)`

```
kmedoids = KMedoids(n_clusters = 4, random_state = 2022)  
kmedoids.fit(X)
```

▼ KMedoids ⓘ
KMedoids(n_clusters=4, random_state=2022)

Obtain the vector at the center of each cluster.

```
kmedoids.cluster_centers_  
array([[ -7.62795904, -8.8354951 ],  
       [ -9.87312876,  0.13931247],  
       [  3.65994783, -0.13260646],  
       [  7.66163195,  2.9333056 ]])
```

```
kmedoids.labels_
```

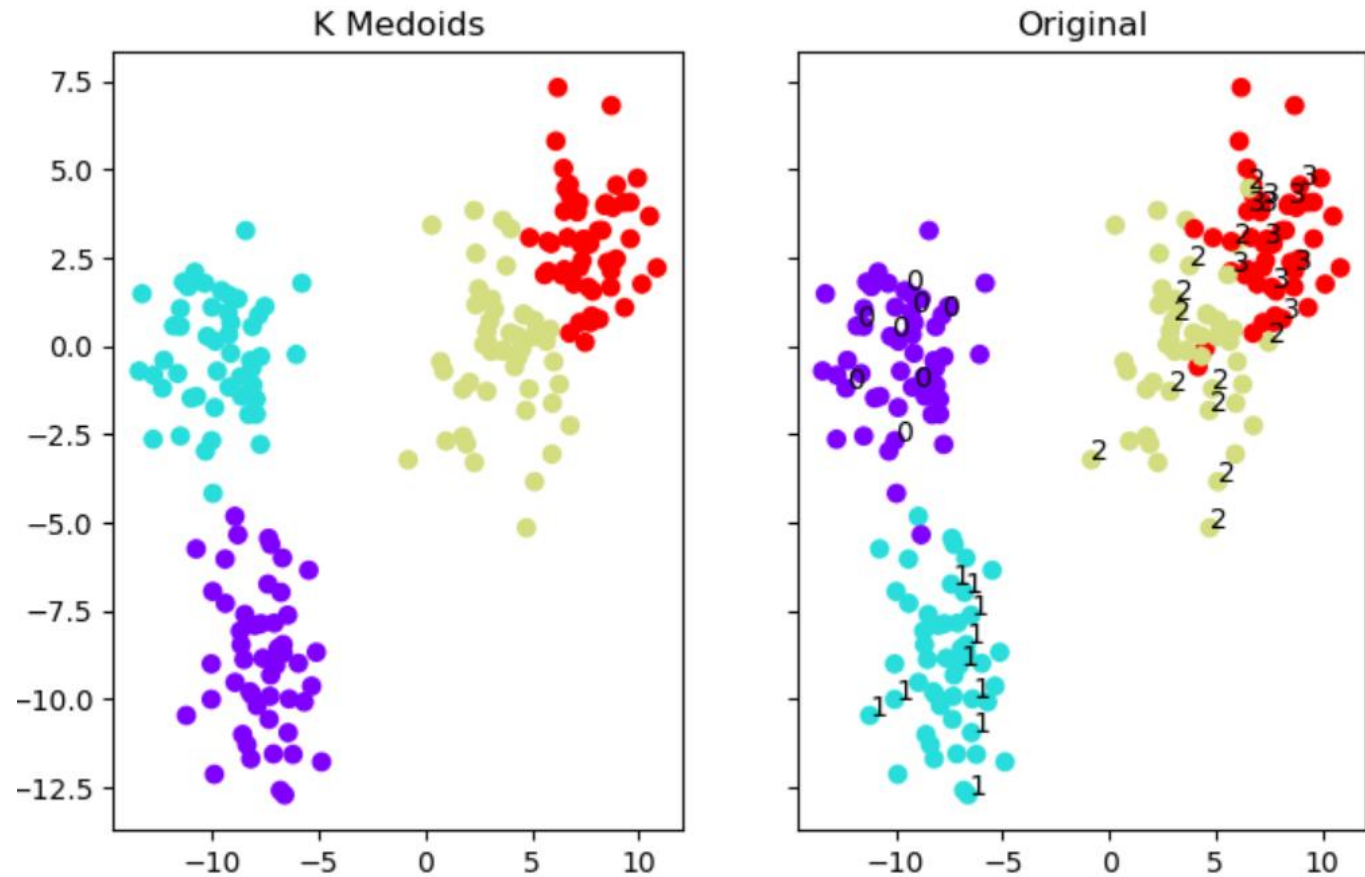
```
array([3, 0, 2, 0, 3, 1, 0, 1, 2, 2, 3, 3, 2, 3, 1, 3, 3, 3, 3, 1, 0, 2,  
       3, 3, 0, 0, 2, 0, 2, 2, 3, 0, 3, 2, 3, 2, 0, 3, 0, 1, 3, 1, 3, 2,  
       2, 2, 0, 0, 2, 0, 0, 2, 3, 3, 0, 2, 3, 2, 3, 1, 1, 1, 2, 2, 3, 1,  
       0, 1, 1, 1, 2, 1, 2, 0, 0, 1, 1, 0, 0, 1, 3, 3, 1, 2, 0, 0, 3, 2,  
       1, 0, 2, 3, 1, 2, 2, 3, 2, 1, 2, 2, 0, 3, 1, 0, 1, 2, 1, 2, 1, 1,  
       3, 1, 1, 1, 2, 1, 2, 2, 0, 2, 3, 3, 3, 2, 0, 0, 3, 2, 1, 3, 0, 1,  
       1, 0, 0, 0, 1, 1, 3, 0, 0, 0, 3, 2, 0, 3, 0, 1, 2, 1, 2, 2, 3, 3,  
       3, 3, 3, 3, 0, 0, 0, 0, 0, 2, 3, 1, 0, 0, 3, 1, 2, 3, 3, 0, 2, 3,  
       2, 0, 0, 1, 2, 0, 1, 1, 0, 1, 3, 3, 1, 2, 1, 2, 2, 0, 1, 3, 1, 0,  
       1, 1], dtype=int64)
```

K-Medoids

23

Visualise the clusters.

```
f, (ax1, ax2) = plt.subplots(1, 2, sharey=True, figsize=(8,5))
ax1.set_title('K Medoids')
ax1.scatter(X[:, 0], X[:, 1], c = kmedoids.labels_, cmap = 'rainbow')
ax2.set_title("Original")
ax2.scatter(X[:, 0], X[:, 1], c = y, cmap = 'rainbow')
for i, txt in enumerate(y):
    if i%5 == 0:
        plt.annotate(txt, (X[i, 0], X[i, 1]))
plt.savefig("KMedoids.png")
```



K-Medoids

24

Align the K-Medoids prediction class with the real values.

```
y_pred = kmedoids.predict(X)
```

y_pred

```
array([3, 0, 2, 0, 3, 1, 0, 1, 2, 2, 3, 3, 2, 3, 1, 3, 3, 3, 3, 1, 0, 2,
       3, 3, 0, 0, 2, 0, 2, 2, 3, 0, 3, 2, 3, 2, 0, 3, 0, 1, 3, 1, 3, 2,
       2, 2, 0, 0, 2, 0, 0, 2, 3, 3, 0, 2, 3, 2, 3, 1, 1, 1, 2, 2, 3, 1,
       0, 1, 1, 1, 2, 1, 2, 0, 0, 1, 1, 0, 0, 1, 3, 3, 1, 2, 0, 0, 3, 2,
       1, 0, 2, 3, 1, 2, 2, 3, 2, 1, 2, 2, 0, 3, 1, 0, 1, 2, 1, 2, 1, 1,
       3, 1, 1, 1, 2, 1, 2, 2, 0, 2, 3, 3, 3, 2, 0, 0, 3, 2, 1, 3, 0, 1,
       1, 0, 0, 0, 1, 1, 3, 0, 0, 0, 3, 2, 0, 3, 0, 1, 2, 1, 2, 2, 3, 3,
       3, 3, 3, 3, 0, 0, 0, 0, 0, 2, 3, 1, 0, 0, 3, 1, 2, 3, 3, 0, 2, 3,
       2, 0, 0, 1, 2, 0, 1, 1, 0, 1, 3, 3, 1, 2, 1, 2, 2, 0, 1, 3, 1, 0,
       1, 1], dtype=int64)
```

y

```
array([2, 1, 2, 1, 3, 0, 1, 0, 2, 2, 3, 3, 2, 3, 0, 3, 3, 3, 3, 0, 1, 2,
       3, 3, 1, 1, 3, 1, 2, 2, 3, 1, 3, 3, 3, 2, 1, 3, 1, 0, 3, 0, 3, 2,
       2, 2, 1, 1, 2, 1, 1, 2, 3, 3, 1, 2, 3, 2, 3, 0, 0, 0, 2, 2, 3, 0,
       1, 0, 0, 0, 2, 0, 2, 1, 1, 0, 0, 1, 1, 0, 2, 3, 0, 2, 1, 1, 3, 2,
       0, 1, 2, 3, 0, 2, 2, 3, 2, 0, 2, 2, 1, 3, 0, 1, 0, 2, 0, 2, 0, 0,
       3, 0, 0, 0, 2, 0, 2, 2, 1, 2, 3, 3, 3, 2, 1, 1, 3, 2, 0, 3, 1, 0,
       0, 1, 1, 1, 0, 0, 3, 1, 1, 1, 3, 2, 1, 3, 1, 0, 2, 0, 2, 2, 3, 3,
       2, 3, 3, 3, 0, 1, 1, 1, 1, 2, 3, 0, 1, 1, 3, 0, 2, 3, 3, 1, 2, 3,
       2, 1, 1, 0, 2, 1, 0, 0, 1, 0, 3, 3, 0, 2, 0, 3, 2, 1, 0, 2, 0, 1,
       0, 0])
```

```
y_pred = np.where(y_pred==1, 10, y_pred)
y_pred = np.where(y_pred==0, 1, y_pred)
y_pred = np.where(y_pred==10, 0, y_pred)
```

y_pred

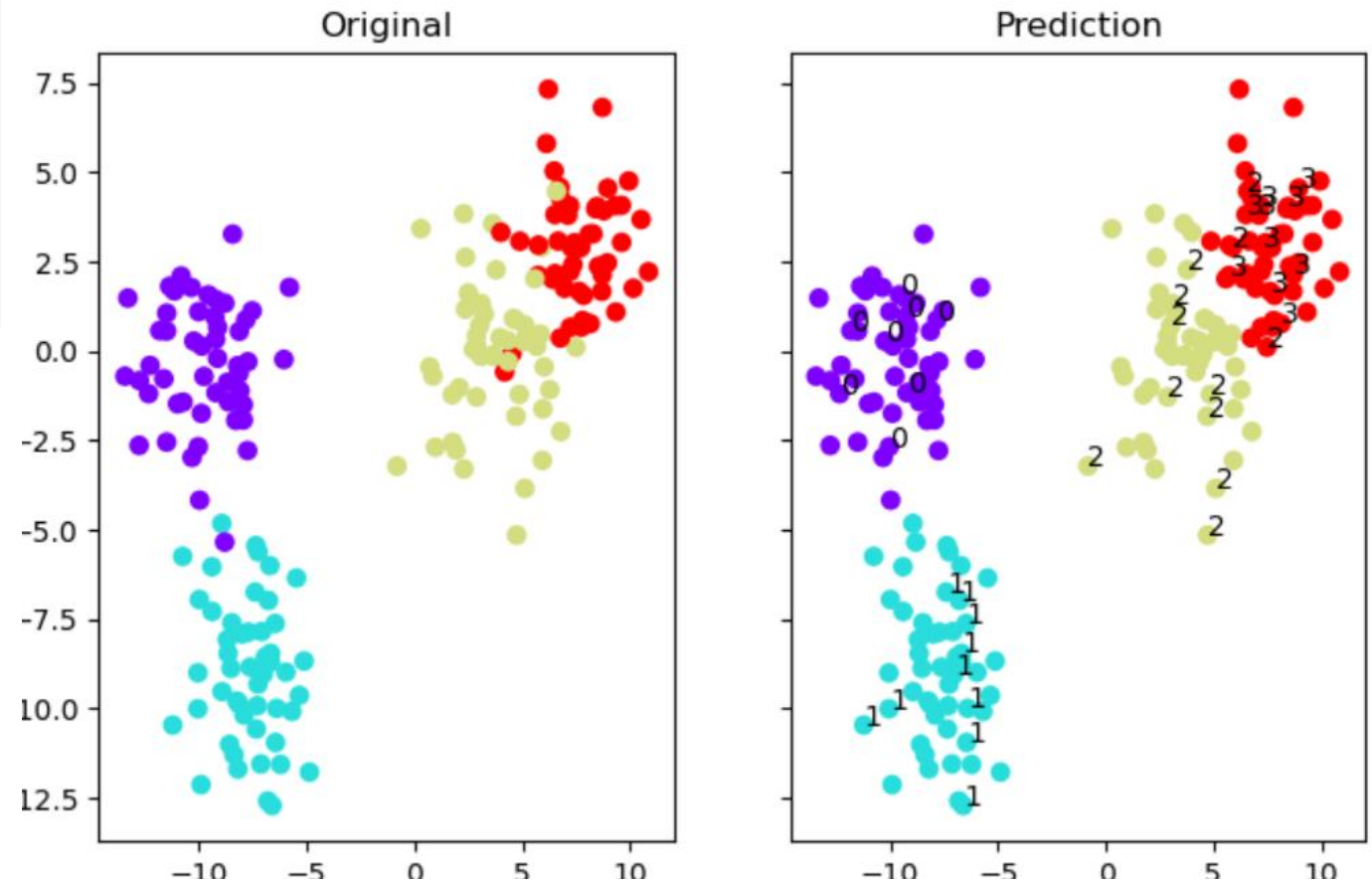
```
array([3, 1, 2, 1, 3, 0, 1, 0, 2, 2, 3, 3, 2, 3, 0, 3, 3, 3, 3, 0, 1, 2,
       3, 3, 1, 1, 2, 1, 2, 2, 3, 1, 3, 2, 3, 2, 1, 3, 1, 0, 3, 0, 3, 2,
       2, 2, 1, 1, 2, 1, 1, 2, 3, 3, 1, 2, 3, 2, 3, 0, 0, 0, 2, 2, 3, 0,
       1, 0, 0, 0, 2, 0, 2, 1, 1, 0, 0, 1, 1, 0, 3, 3, 0, 2, 1, 1, 3, 2,
       0, 1, 2, 3, 0, 2, 2, 3, 2, 0, 2, 2, 1, 3, 0, 1, 0, 2, 0, 2, 0, 0,
       3, 0, 0, 0, 2, 0, 2, 2, 1, 2, 3, 3, 3, 2, 1, 1, 3, 2, 0, 3, 1, 0,
       0, 1, 1, 1, 0, 0, 3, 1, 1, 1, 3, 2, 1, 3, 1, 0, 2, 0, 2, 2, 3, 3,
       3, 3, 3, 3, 1, 1, 1, 1, 1, 2, 3, 0, 1, 1, 3, 0, 2, 3, 3, 1, 2, 3,
       2, 1, 1, 0, 2, 1, 0, 0, 1, 0, 3, 3, 0, 2, 0, 2, 2, 1, 0, 3, 0, 1,
       0, 0], dtype=int64)
```

K-Medoids

25

Redo the visualisations.

```
f, (ax1, ax2) = plt.subplots(1, 2, sharey = True, figsize = (8, 5))
ax1.set_title('Original')
ax1.scatter(X[:, 0], X[:, 1], c = y, cmap = 'rainbow')
ax2.set_title("Prediction")
ax2.scatter(X[:, 0], X[:, 1], c = y_pred, cmap = 'rainbow')
for i, txt in enumerate(y):
    if i%5 == 0:
        plt.annotate(txt, (X[i, 0], X[i, 1]))
plt.savefig("KMedoid_pred.png")
```



K-Medoids

26

Evaluate the model.

```
print(confusion_matrix(y, y_pred))
```

```
[[49  1  0  0]
 [ 0 50  0  0]
 [ 0  0 46  4]
 [ 0  0  3 47]]
```

```
print(classification_report(y, y_pred))
```

	precision	recall	f1-score	support
0	1.00	0.98	0.99	50
1	0.98	1.00	0.99	50
2	0.94	0.92	0.93	50
3	0.92	0.94	0.93	50
accuracy			0.96	200
macro avg	0.96	0.96	0.96	200
weighted avg	0.96	0.96	0.96	200

```
print(f"Homogeneity: {metrics.homogeneity_score(y, y_pred):.3f}")
print(f"Completeness: {metrics.completeness_score(y, y_pred):.3f}")
print(f"V-measure: {metrics.v_measure_score(y, y_pred):.3f}")
print(f"Adjusted Rand Index: {metrics.adjusted_rand_score(y, y_pred):.3f}")
print(f"Adjusted Mutual Information: {metrics.adjusted_mutual_info_score(y, y_pred):.3f}")
print(f"Silhouette Coefficient: {metrics.silhouette_score(X, y_pred):.3f}")
```

Homogeneity: 0.891
Completeness: 0.891
V-measure: 0.891
Adjusted Rand Index: 0.898
Adjusted Mutual Information: 0.889
Silhouette Coefficient: 0.559

K-Means vs. K-Medoids

27

K-Means

```
[[49  1  0  0]
 [ 0 50  0  0]
 [ 0  0 46  4]
 [ 0  0  3 47]]
```

	precision	recall	f1-score	support
0	1.00	0.98	0.99	50
1	0.98	1.00	0.99	50
2	0.94	0.92	0.93	50
3	0.92	0.94	0.93	50
accuracy			0.96	200
macro avg	0.96	0.96	0.96	200
weighted avg	0.96	0.96	0.96	200

Homogeneity: 0.891
Completeness: 0.891
V-measure: 0.891
Adjusted Rand Index: 0.898
Adjusted Mutual Information: 0.889
Silhouette Coefficient: 0.559

K-Medoids

```
[[49  1  0  0]
 [ 0 50  0  0]
 [ 0  0 46  4]
 [ 0  0  3 47]]
```

	precision	recall	f1-score	support
0	1.00	0.98	0.99	50
1	0.98	1.00	0.99	50
2	0.94	0.92	0.93	50
3	0.92	0.94	0.93	50
accuracy			0.96	200
macro avg	0.96	0.96	0.96	200
weighted avg	0.96	0.96	0.96	200

Homogeneity: 0.891
Completeness: 0.891
V-measure: 0.891
Adjusted Rand Index: 0.898
Adjusted Mutual Information: 0.889
Silhouette Coefficient: 0.559

Are these two models comparable?

DBSCAN

28

The **Density-Based Spatial Clustering of Applications with Noise (DBSCAN)** algorithm identifies core samples within high-density regions and the expands the clusters around them.

High-density regions, where data points are located close together, are separated by **low-density regions**, where the data points are located further apart.

A **core sample** is a sample taken from an area of high density. Data point A is considered a core sample if it is located within a distance of ϵ from at least the minimum number of points required to form a dense region (which usually includes A).

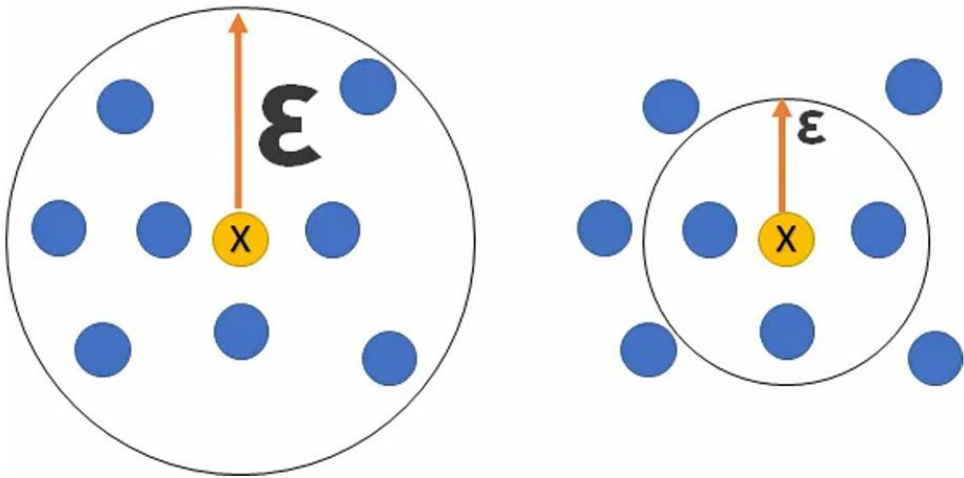
Strengths	Weaknesses
It is not necessary to know the number of clusters	The density parameter (ϵ) needs to be selected and tuned
It solves the anomaly detection task	It does not cope well with sparse data



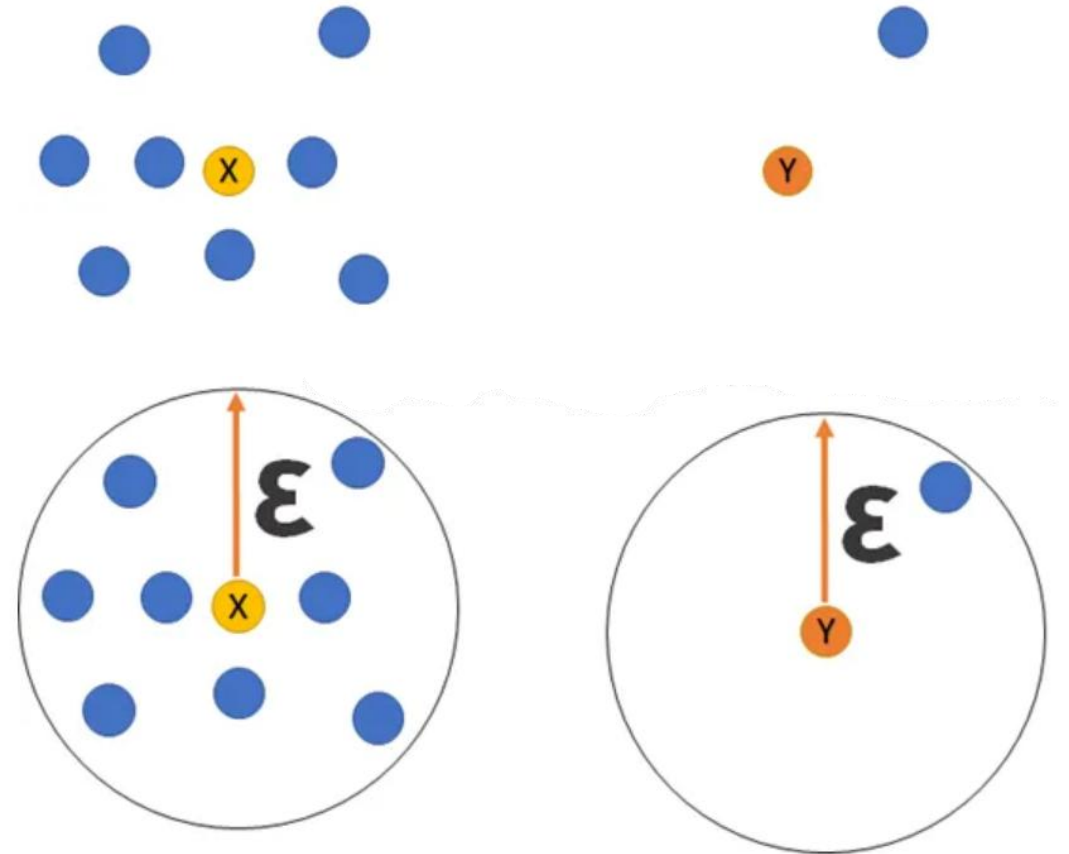
DBSCAN

29

Influence of the **size** of ϵ



Influence of the **density** of the region



The Data

30

Create the dataset.

```
X, y = make_blobs(n_samples=750, cluster_std=0.4, random_state=0)
```

Verify the shapes of X and y, then visualise the data.

```
X.shape
```

```
y.shape
```

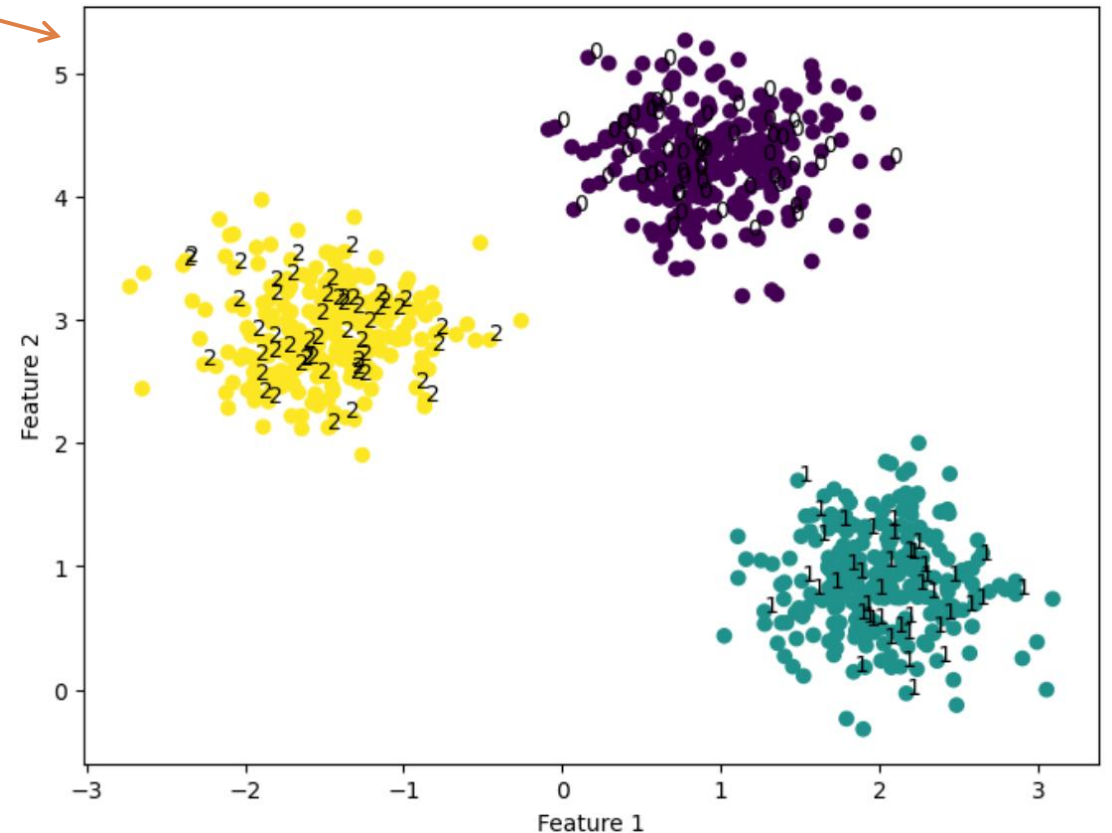
```
(750, 2) (750,)
```

```
print('X:', X[0:5])
print('y:', y[0:5])
```

```
X: [[ 2.36434546  0.23302434]
 [ 0.92311785  4.18467098]
 [ 1.64221028  0.72296432]
 [ 1.97590796  0.93534058]
 [-1.68752703  2.73049184]]
y: [1 0 1 1 2]
```

```
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c = y, cmap = 'viridis')
for i, txt in enumerate(y):
    if i%5 == 0:
        plt.annotate(txt, (X[i, 0], X[i, 1]))
plt.title('Blobs Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

3 groups



DBSCAN

`sklearn.neighbors.NearestNeighbors(*, n_neighbors=5, radius=1.0, algorithm='auto', leaf_size=30, metric='minkowski', p=2, metric_params=None, n_jobs=None)`

31

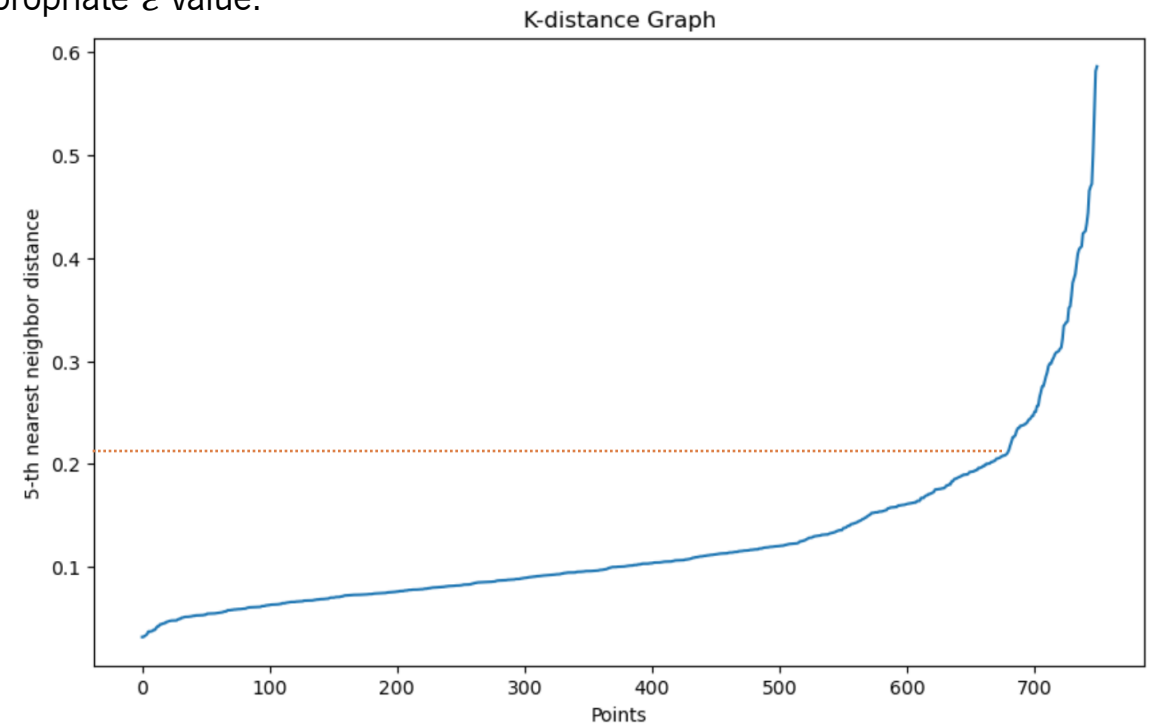
Determining the ε parameter

Select an appropriate ε value using the k-distance graph method:

1. Define a `plot_k_distance_graph` function that calculates the distance to the k^{th} nearest neighbour for each point.
2. These distances are then sorted and plotted.
3. Finally, look for an "elbow" in the resulting graph to select the appropriate ε value.

```
from sklearn.neighbors import NearestNeighbors

def plot_k_distance_graph(X, k):
    neigh = NearestNeighbors(n_neighbors=k)
    neigh.fit(X)
    distances, _ = neigh.kneighbors(X)
    distances = np.sort(distances[:, k-1])
    plt.figure(figsize=(10, 6))
    plt.plot(distances)
    plt.xlabel('Points')
    plt.ylabel(f'{k}-th nearest neighbor distance')
    plt.title('K-distance Graph')
    plt.show()
plot_k_distance_graph(X, k=5)
```



DBSCAN

32

```
from sklearn.cluster import DBSCAN
```

`sklearn.cluster.DBSCAN(eps=0.5, *, min_samples=5, metric='euclidean', metric_params=None, algorithm='auto', leaf_size=30, p=None, n_jobs=None)`

```
db = DBSCAN(eps=0.2, min_samples=10)
```

```
db.fit(X)
```

DBSCAN ⓘ ?

```
DBSCAN(eps=0.2, min_samples=10)
```

Confirm the shape of the labels.

```
labels = db.labels_  
labels.shape
```

```
(750,)
```

labels

```
array([-1,  0,  1,  1,  2, -1,  0, -1,  2, -1,  1, -1,  1, -1,  0,  1,  2,  
       -1,  0, -1,  1,  1,  2,  1,  0,  0,  2,  2,  1,  1,  2,  1,  1,  1,  
        0,  0,  2, -1,  0, -1,  2,  2,  1,  0, -1,  2,  0,  0,  2,  2,  0,  
       -1,  2,  1,  2,  1,  1,  0,  0,  1,  2,  1,  0, -1,  2,  1,  2,  0,  
        0,  2,  2,  2,  1, -1,  1,  2,  2,  1,  2, -1,  1,  0, -1,  0,  2,  
       -1,  0,  0,  2,  2,  2,  0,  1,  1,  0,  2,  1,  2,  0,  0,  0,  0,  
        0,  0,  1,  1,  2,  2,  1,  2,  0,  2,  1,  2,  1,  2,  1,  0,  1,  
        0,  0,  1,  2,  1,  2,  2,  2,  0,  2, -1,  2,  2,  2,  1,  0,  2,  
        0,  0,  0,  0,  1,  2,  2, -1,  1, -1,  2,  1,  1,  1,  2,  1, -1,  
        1,  2,  2,  2, -1,  2,  0,  1,  2,  0,  1,  0,  2,  0,  2,  0,  1,  
        0,  0,  0,  2,  2,  1,  2, -1,  2,  1,  0,  2,  2,  2,  2,  2,  1,  
        2, -1,  2,  0,  1,  2,  0, -1,  2,  1,  1,  1,  0,  2,  0, -1,  2,  
        2,  2,  0,  1,  1,  2,  1,  2,  1, -1,  1,  0, -1,  1,  0, -1,  0,  
       -1,  2,  0,  2,  0,  0,  1,  2,  1,  1,  0,  2,  2,  1,  1,  1, -1,  
        0,  0,  2,  0,  0,  0,  1,  0,  1,  2,  1,  2, -1,  2,  0,  1, -1,  
        2,  1,  2,  1,  0, -1,  2, -1,  0,  0,  2,  0,  1,  0, -1, -1,  2,  
        1,  1,  0,  0,  0,  1,  0,  0,  2,  1,  1, -1,  1,  2,  2,  1,  2,  
        1,  2,  0, -1,  1,  0,  1, -1,  2,  2,  1,  2,  0,  2,  2,  0,  0,  
       -1,  0, -1,  0,  2,  1,  0,  0,  2, -1,  1, -1,  2, -1,  1,  2,  0,
```

DBSCAN

33

Interpreting the results:

Indices of core samples.

```
db.core_sample_indices_  
  
array([ 1,  2,  3,  4,  6,  8, 10, 12, 14, 15, 16, 18, 20,  
       ...,  
       736, 738, 739, 740, 741, 742, 743, 745, 746, 748, 749], dtype=int64)
```

Components.

```
db.components_  
  
array([[0.92311785, 4.18467098],  
       [1.64221028, 0.72296432],  
       [1.97590796, 0.93534058],  
       ...,  
       [1.04724054, 4.14307495],  
       [2.16897939, 1.59473117],  
       [0.73774446, 4.28276041]])
```

Number of features.

```
db.n_features_in_  
  
2
```

The number of clusters and noise points in the labels.

```
n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)  
n_noise_ = list(labels).count(-1)
```

```
print("Estimated number of clusters: %d" % n_clusters_)  
print("Estimated number of noise points: %d" % n_noise_)
```

```
Estimated number of clusters: 3  
Estimated number of noise points: 104
```

DBSCAN

34

Obtain the predictions..

```
pred = db.fit_predict(X)
```

pred

```
array([-1,  0,  1,  1,  2, -1,  0, -1,  2, -1,  1, -1,  1, -1,  0,  1,  2,
       -1,  0, -1,  1,  1,  2,  1,  0,  0,  2,  2,  1,  1,  2,  1,  1,  1,
        0,  0,  2, -1,  0, -1,  2,  2,  1,  0, -1,  2,  0,  0,  2,  2,  0,
       -1,  2,  1,  2,  1,  1,  0,  0,  1,  2,  1,  0, -1,  2,  1,  2,  0,
        0,  2,  2,  2,  1, -1,  1,  2,  2,  1,  2, -1,  1,  0, -1,  0,  2,
       -1,  0,  0,  2,  2,  2,  0,  1,  1,  0,  2,  1,  2,  0,  0,  0,  0,
        0,  0,  1,  1,  2,  2,  1,  2,  0,  2,  1,  2,  1,  2,  1,  0,  1,
        0,  0,  1,  2,  1,  2,  2,  2,  0,  2, -1,  2,  2,  2,  1,  0,  2,
        0,  0,  0,  0,  1,  2,  2, -1,  1, -1,  2,  1,  1,  1,  2,  1, -1,
        1,  2,  2,  2, -1,  2,  0,  1,  2,  0,  1,  0,  2,  0,  2,  0,  1,
        0,  0,  0,  2,  2,  1,  2, -1,  2,  1,  0,  2,  2,  2,  2,  2,  1,
        2, -1,  2,  0,  1,  2,  0, -1,  2,  1,  1,  1,  0,  2,  0, -1,  2,
        2,  2,  0,  1,  1,  2,  1,  2,  1, -1,  1,  0, -1,  1,  0, -1,  0,
       -1,  2,  0,  2,  0,  0,  1,  2,  1,  1,  0,  2,  2,  1,  1,  1, -1,
        0,  0,  2,  0,  0,  0,  1,  0,  1,  2,  1,  2, -1,  2,  0,  1, -1,
        2,  1,  2,  1,  0, -1,  2, -1,  0,  0,  2,  0,  1,  0, -1, -1,  2,
        1,  1,  0,  0,  0,  1,  0,  0,  2,  1,  1, -1,  1,  2,  2,  1,  2,
        1,  2,  0, -1,  1,  0,  1, -1,  2,  2,  1,  2,  0,  2,  2,  0,  0,
       -1,  0, -1,  0,  2,  1,  0,  0,  2, -1,  1, -1,  2, -1,  1,  2,  0]
```

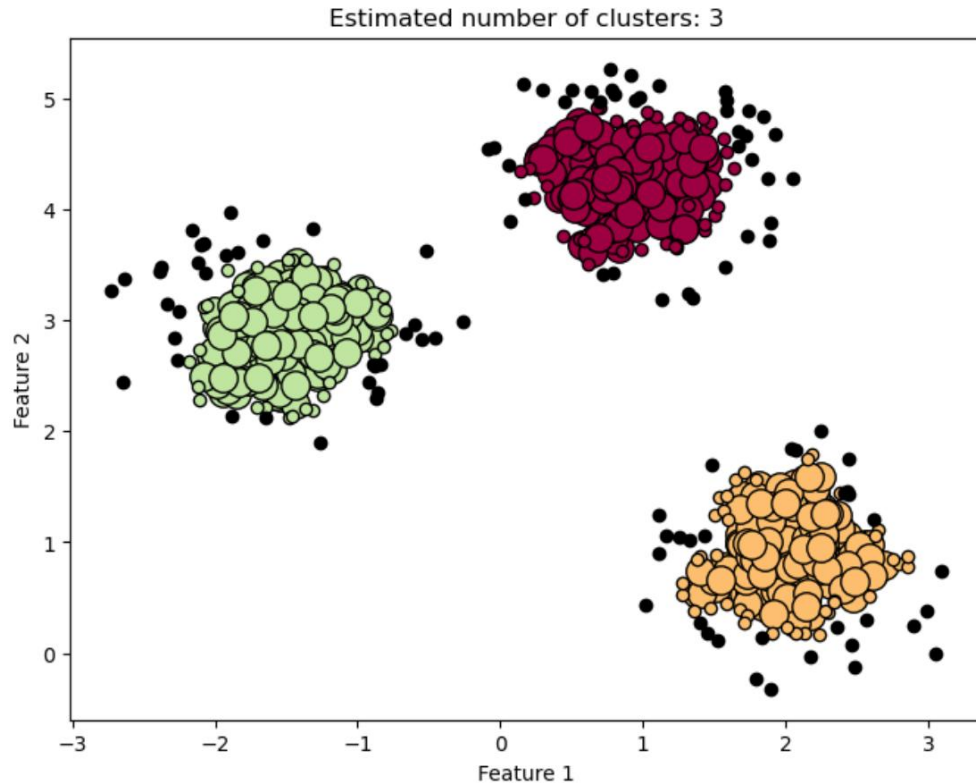
y

```
array([[1,  0,  1,  1,  2,  2,  0,  0,  2,  2,  1,  0,  1,  0,  0,  1,  2,  1,  0,  2,  1,  1,
        2,  1,  0,  0,  2,  2,  1,  1,  2,  1,  1,  1,  0,  0,  2,  2,  0,  2,  2,  2,  1,  0,
        0,  2,  0,  0,  2,  2,  0,  2,  2,  1,  2,  1,  1,  0,  0,  1,  2,  1,  0,  1,  2,  1,
        2,  0,  0,  2,  2,  2,  1,  0,  1,  2,  2,  1,  2,  0,  1,  0,  0,  0,  2,  1,  0,  0,
        2,  2,  2,  0,  1,  1,  0,  2,  1,  2,  0,  0,  0,  0,  0,  0,  1,  1,  2,  2,  1,  2,
        0,  2,  1,  2,  1,  2,  1,  0,  1,  0,  0,  1,  2,  1,  2,  2,  2,  0,  2,  1,  2,  2,
        2,  1,  0,  2,  0,  0,  0,  0,  1,  2,  2,  1,  1,  0,  2,  1,  1,  1,  2,  1,  0,  1,
        2,  2,  2,  0,  2,  0,  1,  2,  0,  1,  0,  2,  0,  2,  0,  1,  0,  0,  0,  2,  2,  1,
        2,  2,  2,  1,  0,  2,  2,  2,  2,  1,  2,  2,  2,  0,  1,  2,  0,  0,  2,  1,  1,
        1,  0,  2,  0,  1,  2,  2,  2,  0,  1,  1,  2,  1,  2,  1,  2,  1,  0,  0,  1,  0,  1,
        0,  0,  2,  0,  2,  0,  0,  1,  2,  1,  1,  0,  2,  2,  1,  1,  1,  1,  0,  0,  2,  0,
        0,  0,  1,  0,  1,  2,  1,  2,  2,  2,  0,  1,  2,  2,  1,  2,  1,  0,  1,  2,  1,  0,
        0,  2,  0,  1,  0,  2,  2,  2,  1,  1,  0,  0,  0,  1,  0,  0,  2,  1,  1,  2,  1,  2,
        2,  1,  2,  1,  2,  0,  1,  1,  0,  1,  1,  2,  2,  1,  2,  0,  2,  2,  0,  0,  2,  0,
        2,  0,  2,  1,  0,  0,  2,  0,  1,  1,  2,  1,  1,  2,  0,  2,  1,  0,  0,  2,  1,  2,
        2,  1,  2,  0,  1,  2,  1,  1,  0,  0,  1,  1,  1,  0,  2,  0,  1,  2,  2,  0,  0,  2,
        0,  2,  1,  0,  2,  0,  2,  0,  2,  1,  0,  2,  2,  0,  2,  0,  1,  2,  1,  1,  1,  1,
        2,  0,  2,  0,  2,  1,  2,  0,  0,  2,  0,  1,  0,  0,  1,  0,  0,  1,  0,  2,  0,  1,
```

DBSCAN

35

Visualise the resulting clusters.



```
unique_labels = set(labels)
core_samples_mask = np.zeros_like(labels, dtype=bool)
core_samples_mask[db.core_sample_indices_] = True

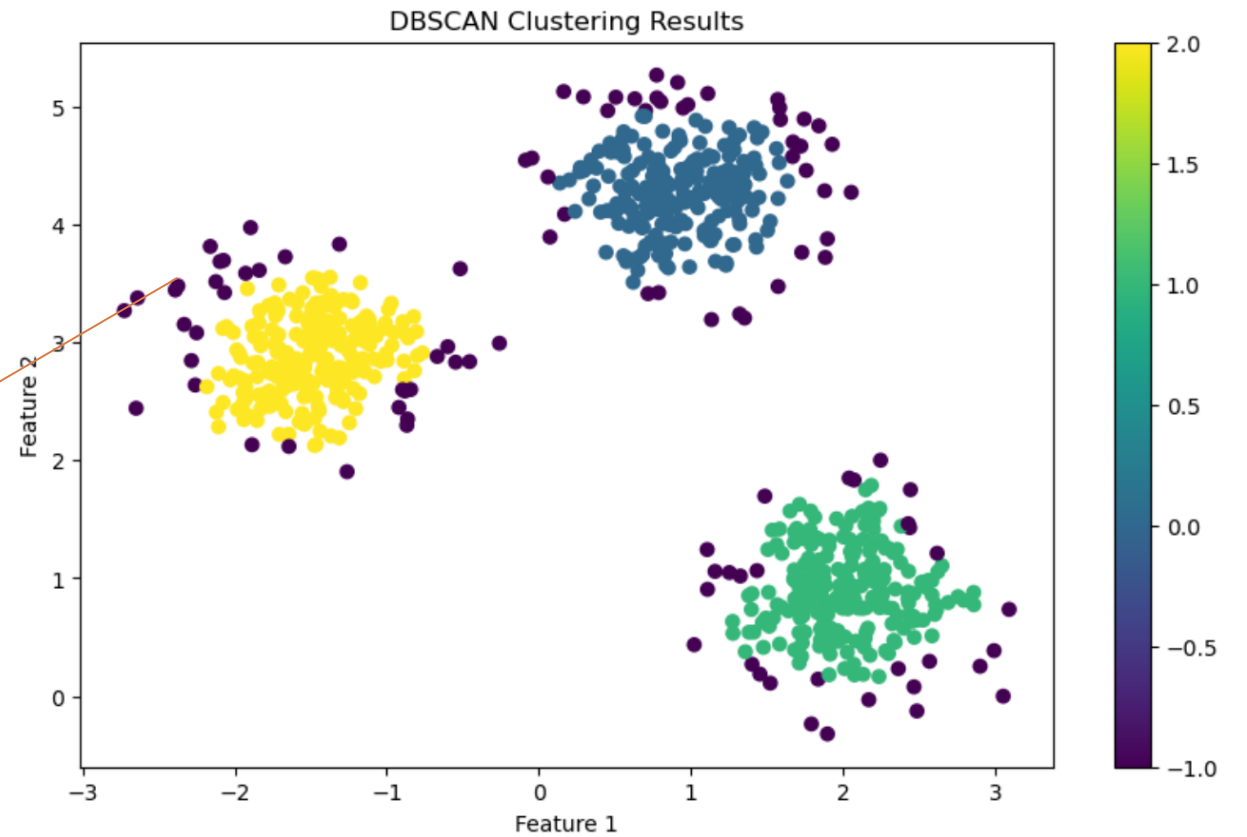
plt.figure(figsize=(8, 6))
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]
for k, col in zip(unique_labels, colors):
    if k == -1:
        col = [0, 0, 0, 1]
    class_member_mask = labels == k
    xy = X[class_member_mask & core_samples_mask]
    plt.plot(
        xy[:, 0],
        xy[:, 1],
        "o",
        markerfacecolor=tuple(col),
        markeredgecolor="k",
        markersize=14,)
    xy = X[class_member_mask & ~core_samples_mask]
    plt.plot(
        xy[:, 0],
        xy[:, 1],
        "o",
        markerfacecolor=tuple(col),
        markeredgecolor="k",
        markersize=6,)
plt.title(f"Estimated number of clusters: {n_clusters_}")
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

DBSCAN

36

```
plt.figure(figsize=(10, 6))
scatter = plt.scatter(X[:, 0], X[:, 1], c=pred, cmap='viridis')
plt.colorbar(scatter)
plt.title('DBSCAN Clustering Results')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

anomalies detection: outliers

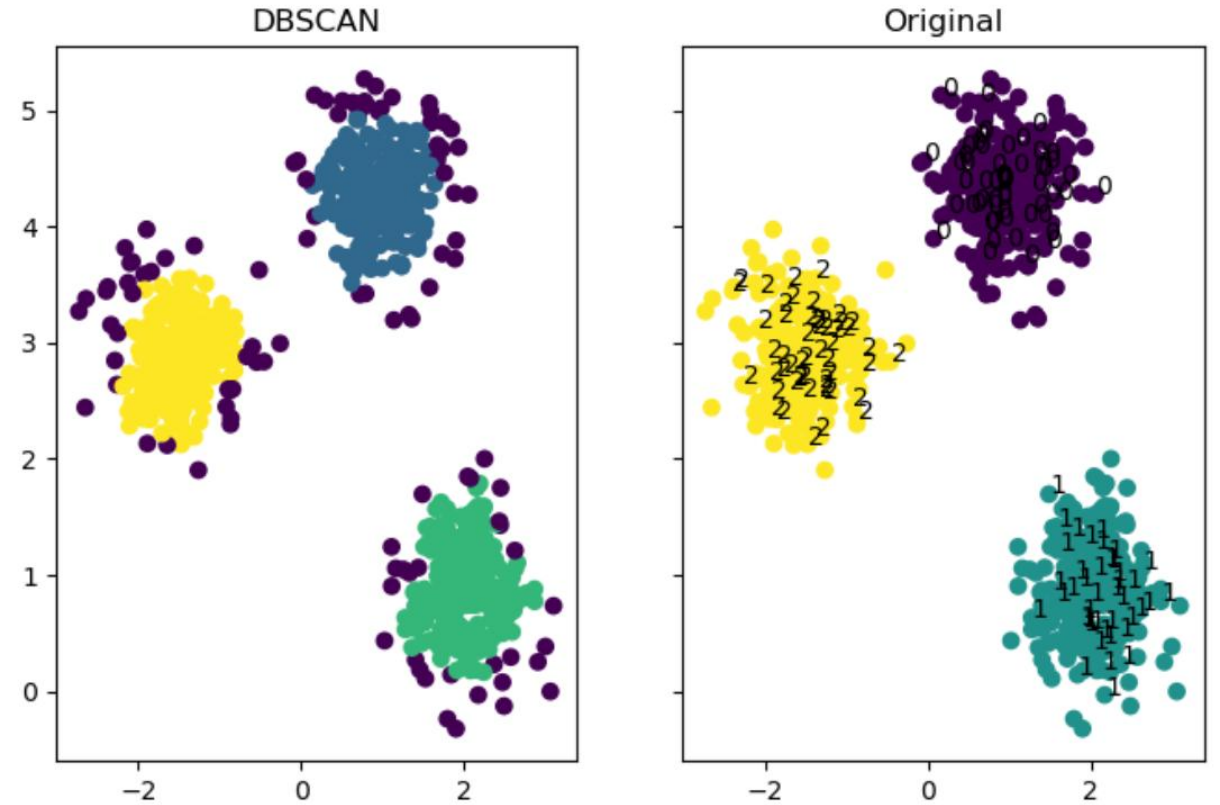


DBSCAN

37

Visualise the original clusters versus the DBSCAN clusters.

```
f, (ax1, ax2) = plt.subplots(1, 2, sharey=True, figsize=(8,5))
ax1.set_title('DBSCAN')
ax1.scatter(X[:, 0], X[:, 1], c = db.labels_, cmap = 'viridis')
ax2.set_title("Original")
ax2.scatter(X[:, 0], X[:, 1], c = y, cmap = 'viridis')
for i, txt in enumerate(y):
    if i%5 == 0:
        plt.annotate(txt, (X[i, 0], X[i, 1]))
plt.savefig("DBSCAN.png")
plt.show()
```



DBSCAN

38

```
print(f"Homogeneity: {metrics.homogeneity_score(y, labels):.3f}")
print(f"Completeness: {metrics.completeness_score(y, labels):.3f}")
print(f"V-measure: {metrics.v_measure_score(y, labels):.3f}")
print(f"Adjusted Rand Index: {metrics.adjusted_rand_score(y, labels):.3f}")
print(f"Adjusted Mutual Information: {metrics.adjusted_mutual_info_score(y, labels):.3f}")
print(f"Silhouette Coefficient: {metrics.silhouette_score(X, labels):.3f}")
```

Homogeneity: 0.862
Completeness: 0.702
V-measure: 0.774
Adjusted Rand Index: 0.781
Adjusted Mutual Information: 0.773
Silhouette Coefficient: 0.586

```
print(confusion_matrix(y, pred))
```

```
[[ 0  0  0  0]
 [39 211  0  0]
 [31  0 219  0]
 [34  0  0 216]]
```

```
print(classification_report(y, pred))
```

	precision	recall	f1-score	support
-1	0.00	0.00	0.00	0
0	1.00	0.84	0.92	250
1	1.00	0.88	0.93	250
2	1.00	0.86	0.93	250
accuracy			0.86	750
macro avg	0.75	0.65	0.69	750
weighted avg	1.00	0.86	0.93	750

DBSCAN vs. K-Means

39

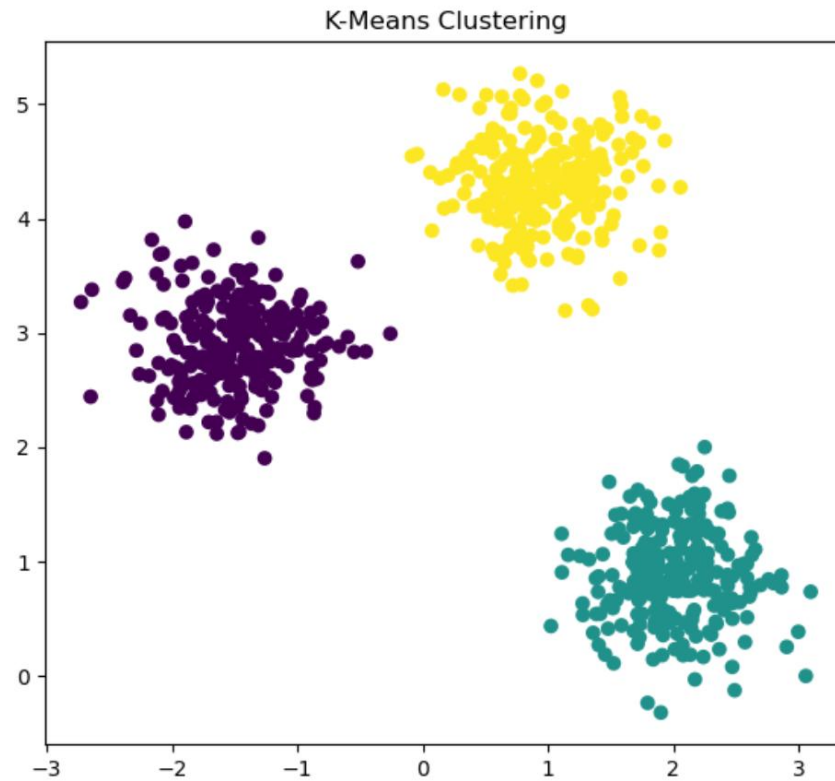
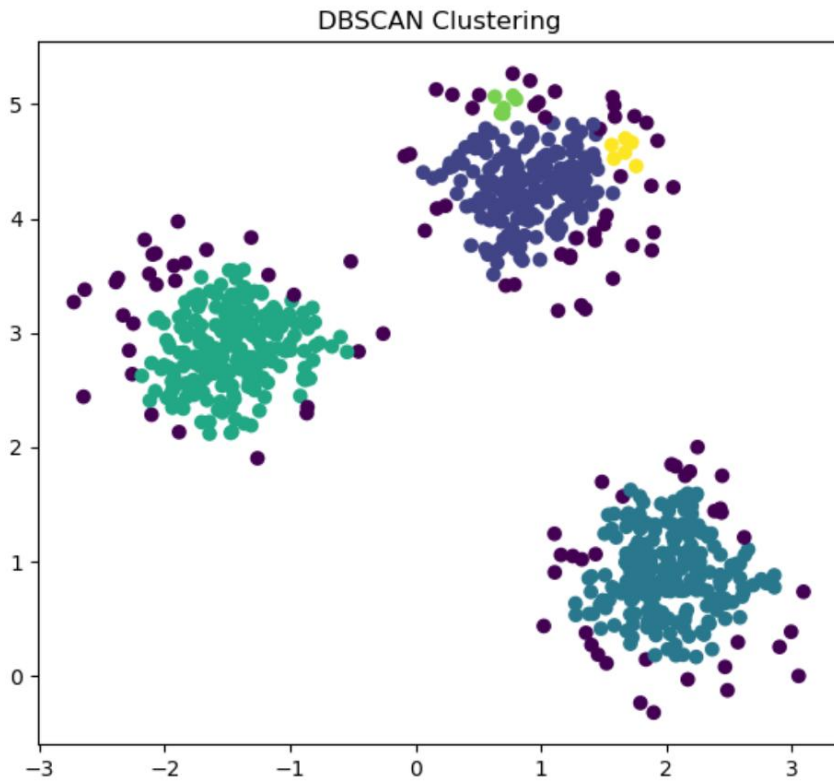
Comparing DBSCAN and K-Means:

```
dbscan = DBSCAN(eps=0.15, min_samples=5)
dbscan_labels = dbscan.fit_predict(X)
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
ax1.scatter(X[:, 0], X[:, 1], c=dbscan_labels, cmap='viridis')
ax1.set_title('DBSCAN Clustering')
ax2.scatter(X[:, 0], X[:, 1], c=kmeans_labels, cmap='viridis')
ax2.set_title('K-Means Clustering')
plt.show()
```

DBSCAN vs. K-Means

40

Comparing DBSCAN and K-Means:



DBSCAN vs. K-Means

41

Comparing DBSCAN and K-Means:

Classification report

DBSCAN: precision recall f1-score support

-1	0.00	0.00	0.00	0
0	1.00	0.78	0.88	250
1	1.00	0.86	0.92	250
2	1.00	0.88	0.94	250
3	0.00	0.00	0.00	0
4	0.00	0.00	0.00	0

accuracy			0.84	750
macro avg	0.50	0.42	0.46	750
weighted avg	1.00	0.84	0.91	750

K-Means: precision recall f1-score support

0	0.00	0.00	0.00	250
1	1.00	1.00	1.00	250
2	0.00	0.00	0.00	250

accuracy			0.33	750
macro avg	0.33	0.33	0.33	750
weighted avg	0.33	0.33	0.33	750

Homogeneity

DBSCAN: 0.857

K-Means: 1.000

Completeness

DBSCAN: 0.661

K-Means: 1.000

V-measure

DBSCAN: 0.747

K-Means: 1.000

Adjusted Rand Index

DBSCAN: 0.751

K-Means: 1.000

Adjusted Mutual Information:

DBSCAN: 0.745

K-Means: 1.000

Silhouette Coefficient

DBSCAN: 0.413

K-Means: 0.771

Confusion Matrix

DBSCAN

```
[[ 0  0  0  0  0  0]
 [43 195  0  0  6  6]
 [35  0 215  0  0  0]
 [30  0  0 220  0  0]
 [ 0  0  0  0  0  0]
 [ 0  0  0  0  0  0]]
```

K-Means

```
[[ 0  0 250]
 [ 0 250  0]
 [250  0  0]]
```

What about the labels?

DBSCAN vs. K-Means

42

Comparing DBSCAN and K-Means (with corrected labels):

Classification report

DBSCAN:	precision	recall	f1-score	support
-1	0.00	0.00	0.00	0
0	1.00	0.78	0.88	250
1	1.00	0.86	0.92	250
2	1.00	0.88	0.94	250
3	0.00	0.00	0.00	0
4	0.00	0.00	0.00	0
accuracy			0.84	750
macro avg	0.50	0.42	0.46	750
weighted avg	1.00	0.84	0.91	750

K-Means:	precision	recall	f1-score	support
0	1.00	1.00	1.00	250
1	1.00	1.00	1.00	250
2	1.00	1.00	1.00	250
accuracy			1.00	750
macro avg	1.00	1.00	1.00	750
weighted avg	1.00	1.00	1.00	750

Homogeneity

DBSCAN: 0.857
K-Means: 1.000

Completeness

DBSCAN: 0.661
K-Means: 1.000

V-measure

DBSCAN 0.747
K-Means: 1.000

Adjusted Rand Index

DBSCAN: 0.751
K-Means: 1.000

Adjusted Mutual Information:

DBSCAN: 0.745
K-Means: 1.000

Silhouette Coefficient

DBSCAN: 0.413
K-Means: 0.771

Confusion Matrix

DBSCAN

```
[[ 0  0  0  0  0  0]
 [43 195  0  0  6  6]
 [35  0 215  0  0  0]
 [30  0  0 220  0  0]
 [ 0  0  0  0  0  0]
 [ 0  0  0  0  0  0]]
```

K-Means

```
[[250  0  0]
 [ 0 250  0]
 [ 0  0 250]]
```

What conclusions can be drawn?



Hands On